

北京市住房价格影响因素分析

——基于特征价格模型的多重回归实证研究

摘 要

本研究基于链家平台 2011 至 2017 年北京市二手房交易数据，在特征价格模型框架下，综合运用 OLS 回归、Ridge 回归、LASSO 回归及多项式扩展等方法，系统考察了影响北京市住房价格的关键因素。空间因素（距市中心距离、经纬度、区域虚拟变量）是解释房价差异的主导变量，呈现显著的单中心城市空间衰减格局。交易年份对价格具有强预测能力且呈明显非线性，房屋属性（电梯、地铁、梯户比）的边际效应在统计和经济意义上均显著。在模型比较层面，多项式模型预测精度最高（ $R^2 = 0.829$ ），但高阶项引发的多重共线性严重损害了系数的可解释性。综合预测精度与经济可解释性，本文推荐以纳入区域和时间双向固定效应的线性主效应模型作为结论支撑。

关键词：特征价格模型；住房价格；多重共线性；LASSO 回归；稳健标准误；固定效应

1. 引言

住房兼具消费与投资属性，其价格形成机制是城市经济学的核心议题。北京市作为中国政治文化中心，2011 至 2017 年间二手房均价从约 2.5 万元/ m^2 上升至约 6 万元/ m^2 。这一时期伴随快速城镇化、宽松货币政策和调控政策（如「京十五条」「9·30 新政」等）的交替作用，呈现出典型的政策周期性特征。住房价格的剧烈波动直接影响居民购房决策、财富分配与城市空间结构演化，准确识别房价的影响因素及其边际效应对房地产市场调控政策的科学制定具有重要的现实意义。

特征价格模型是住房价格研究的经典框架，由 Rosen[1] 在 Lancaster[2] 消费者理论基础上系统发展而来。该模型的核心思想是：异质性商品的价格由其特征隐含价格决定。在实证方法层面，LASSO 回归 [3] 通过 L1 惩罚实现自动变量选择，Breusch-Pagan 检验 [4] 和 White 稳健标准误 [5] 已成为异方差处理的标准工具。国内方面，郑思齐等 [7] 较早将特征价格模型应用于北京住房市场。然而，已有研究较少在同一框架内系统比较多种回归方法的表现差异，也往往忽视了高缺失率变量的处理偏倚和复杂模型的可解释性损失。

本研究基于链家北京 2011–2017 年微观交易数据，构建了从数据预处理、描述统计分析、变量选择、模型比较到模型诊断的完整回归分析流程。主要贡献体现在三个方面。第一，系统比较了六种回归模型在预测精度与经济可解释性之间的权衡。第二，分析了多项式模型因多重共线性引发的系数膨胀问题。第三，强调核心变量的半弹性解释，在预测精度与可解释性之间寻求恰当的平衡。

2. 数据来源与预处理

2.1 数据来源与变量定义

本研究使用链家北京二手房交易数据（2011–2017 年），来源于 Kaggle 开放数据平台（<https://www.kaggle.com/datasets/ruiqurm/lianjia>），收录了链家网（bj.lianjia.com）公开的北京二手房成交记录，涵盖交易价格、房屋属性、地理位置及社区特征等多维信息。原始数据包含 318 621 条交易记录和 26 个变量。表 1 列出了主要变量及其含义。

表 1: 变量定义

变量名	含义	类型
<i>price</i>	每平方米价格（元/m ² ）	响应变量
<i>log_price</i>	对数价格（建模目标）	响应变量
<i>dist_center</i>	距天安门距离（度）	数值
<i>Lng, Lat</i>	经度、纬度	数值
<i>square</i>	建筑面积（m ² ）	数值
<i>trade_year</i>	交易年份	数值
<i>elevator</i>	有无电梯（1/0）	二元虚拟
<i>subway</i>	临近地铁（1/0）	二元虚拟
<i>ladderRatio</i>	梯户比	数值
<i>DOM</i>	挂牌天数	数值
<i>district_*</i>	所在区域（15 类）	分类虚拟

2.2 数据预处理

数据预处理包括四个步骤。第一，剔除数据泄露变量（社区均价、总价等）和标识符变量，修正数据类型错误。第二，过滤极端值（价格 <100 元/m²、面积 >1 000m² 等），共删除约 300 行。第三，处理缺失值：*DOM* 缺失率达 50%，采用中位数填充并添加缺失指示变量 *DOM_missing*；*constructionTime* 缺失约 6%，用中位数填充。第四，特征工程：对 *price* 取自然对数作为回归目标（偏度从 1.31 降至 -0.59），对 10 个分类变量进行独热编码，生成时间固定效应（6 个年度虚拟变量 +11 个月度虚拟变量）。最终特征空间包含 16 个数值型变量和 48 个虚拟变量，共计 64 个特征。

2.3 训练-测试集划分

将数据按 80/20 比例随机划分为训练集（254 528 条）和测试集（63 632 条）。数值型特征经标准化处理后用于 Ridge 和 LASSO 正则化模型，虚拟变量不进行标准化。

3. 描述统计统计分析

表 2 报告了关键变量的描述统计结果。原始价格均值为 43 562 元/m²，中位数为 38 752 元/m²，偏度为 1.31，呈明显右偏分布，高价位存在长尾。对数变换后偏度降至 -0.59，分布趋近对称。样本中有电梯的房源占 57.7%，临近地铁的房源占 60.1%，满五年的房源占 64.6%。梯户比中位数为 0.33，挂牌天数中位数为 6 天，反映多数房源在短期内成交的特征。

表 2: 关键变量描述统计

变量	均值	标准差	最小值	中位数	最大值
单价（元/m ² ）	43,561.9	21,685.3	117.0	38,752.0	156,250.0
对数单价	10.6	0.5	4.8	10.6	12.0
距市中心距离（度）	0.1	0.1	0.0	0.1	0.5
建筑面积（m ² ）	83.2	37.2	6.9	74.3	1,745.5
经度	116.4	0.1	116.1	116.4	116.7
纬度	39.9	0.1	39.6	39.9	40.3
梯户比	0.4	0.2	0.1	0.3	1.0
挂牌天数	28.8	50.2	1.0	6.0	1,677.0
建造年份	1,999.0	22.5	0.0	2,001.0	2,016.0
有电梯（比例）	0.577	0.494	0	1	1
临近地铁（比例）	0.601	0.490	0	1	1
满五年（比例）	0.646	0.478	0	1	1

图 1 进一步展示了价格分布与对数变换效果。原始价格偏度为 1.31，呈明显右偏；对数变换后偏度降至 -0.59，分布接近对称，验证了对数变换的合理性。Q-Q 图尾部的偏离表明对数正态假设仍有不足，为后文使用 HC3 稳健标准误提供了依据。

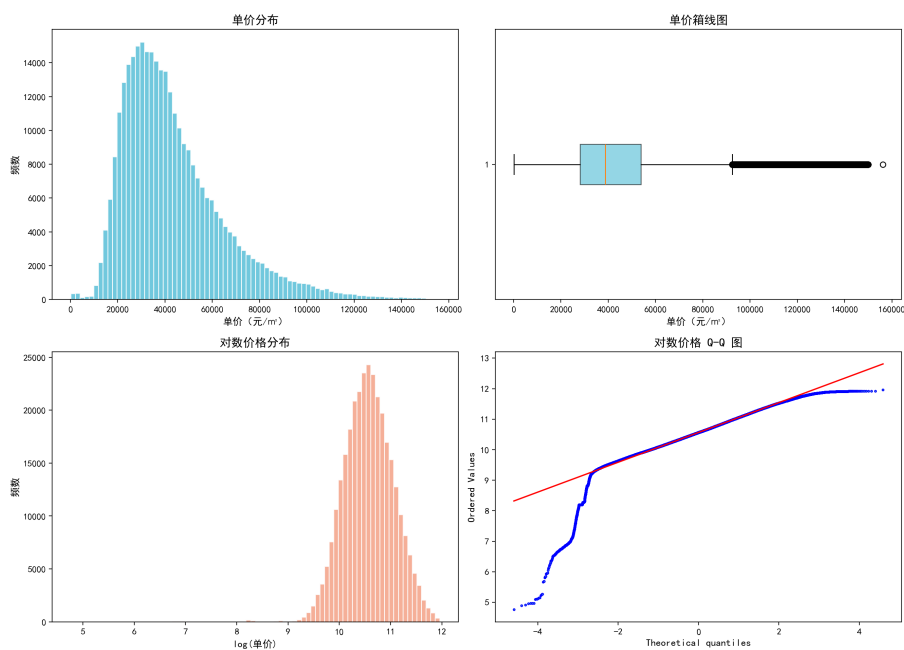


图 1: 价格分布与对数变换对比

图 2 为相关性热力图。*livingRoom* 与 *square* 相关系数达 0.72，存在多重共线性隐患。剔除数据泄露变量后，与价格相关性最高的变量依次为经纬度、面积和建造年份。

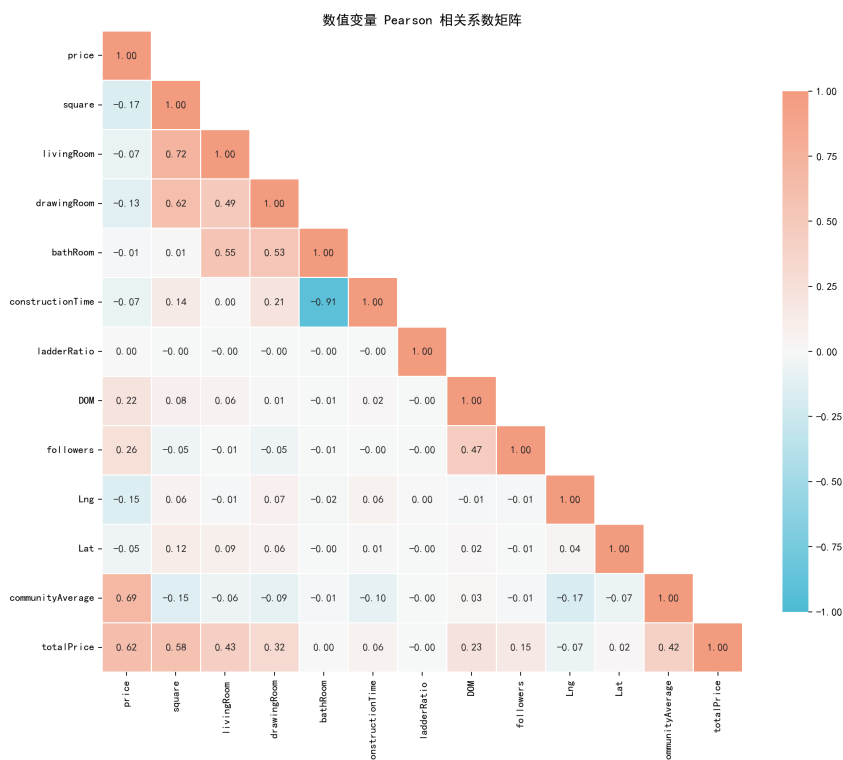


图 2: 相关性热力图

图 3 显示房价地理分布呈现典型的单中心空间结构：颜色梯度从市中心向外围辐射状衰减，东城、西城和朝阳核心区价格最高，远郊区显著偏低，价差达三到四倍。

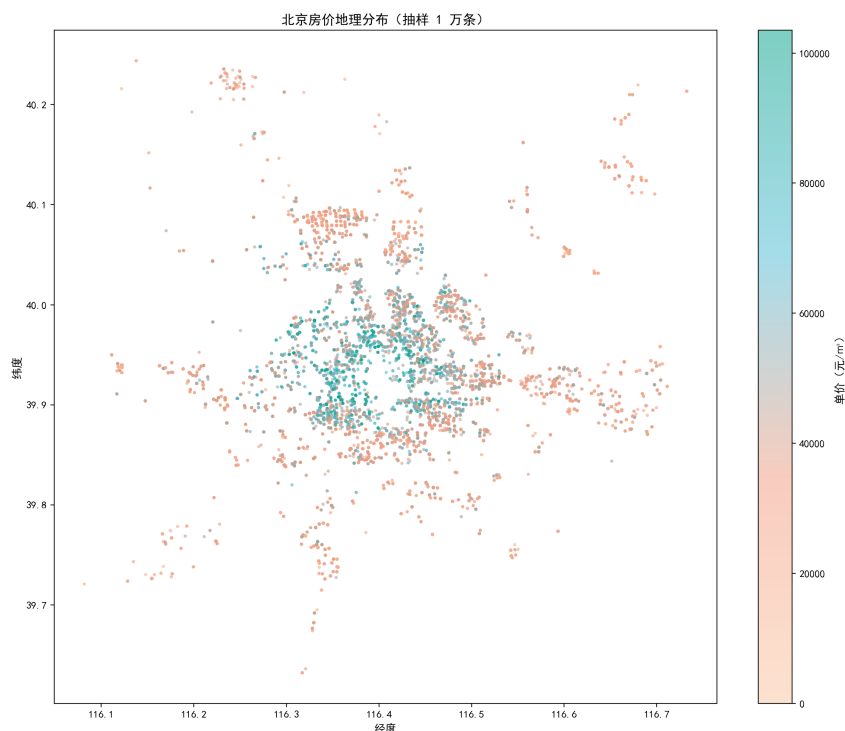


图 3: 房价地理分布散点图

图 4 从区域维度量化了上述空间差异。东城、西城均价超过 7 万元/ m^2 ，朝阳、海淀紧随其后（约 5-6 万元/ m^2 ），而房山、平谷等远郊区均价不足 3 万元/ m^2 ，最高与最低区域价差接近三倍。各区域样本量差异较大，朝阳区交易量最大（超过 6 万条），平谷区最少（约 3000 条），反映了北京二手房市场的活跃度在空间上的不均匀分布。

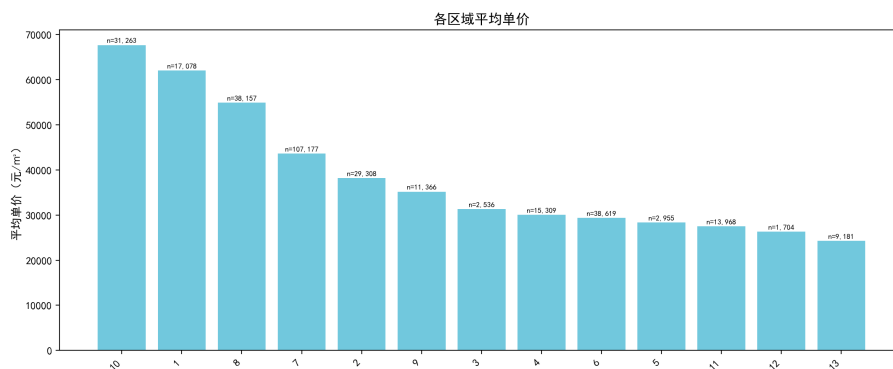


图 4: 各区域平均单价对比

图 5 为分年度价格趋势。2011-2015 年初为平缓上涨阶段，2015 年下半年后加速攀升，均价从约 4 万元/ m^2 跃升至约 7 万元/ m^2 。这一非线性趋势表明，将 *trade_year* 作为连续变量会施加「年增幅恒定」的不当约束。因此，本文采用年份虚拟变量替代，让数据自行估计每年的基准价格水平。

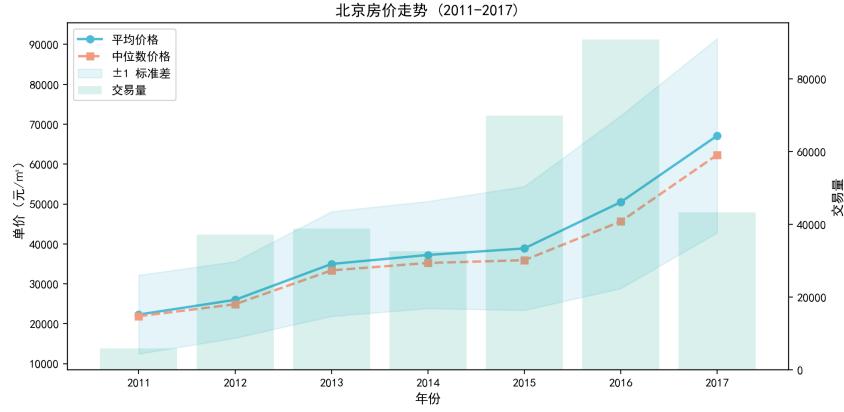


图 5: 分年度价格趋势

4. 模型设定与变量选择

4.1 基准回归模型

本研究在特征价格模型框架下，设定包含时间和区域双向固定效应的半对数回归方程作为基准模型：

$$\log(\text{Price}_{it}) = \beta_0 + \mathbf{X}'_{it}\boldsymbol{\beta} + \gamma_t + \delta_k + \varepsilon_{it} \quad (1)$$

其中 $\log(\text{Price}_{it})$ 为第 i 条交易在第 t 期的对数价格， $\mathbf{X}_{it}$ 包含 dist_center （距市中心距离）、 Lat （纬度）、 Lng （经度）、 square （建筑面积）、 ladderRatio （梯户比）、 DOM （挂牌天数）等连续型房屋和区位特征。 γ_t 为时间固定效应，由年度虚拟变量（ trade_year_2012 至 trade_year_2017 ，以 2011 年为基期）和月度虚拟变量（ trade_month_2 至 trade_month_12 ，以 1 月为基期）共同实现。 δ_k 为区域固定效应。

该设定的关键优势在于不施加年度价格增幅恒定的线性约束，允许数据自行估计每年的基准价格水平。第三节的描述统计分析已表明 2011–2017 年房价趋势呈明显非线性特征，2015 年前平缓上涨、2015 年后加速攀升，因此 γ_t 的灵活设定远优于线性趋势变量。月度虚拟变量则进一步控制季节性波动。

为探索空间和房屋属性变量的非线性边际效应，对 5 个关键连续变量构造多项式扩展模型：

$$\log(\text{Price}) = \beta_0 + \mathbf{X}'\boldsymbol{\beta} + \mathbf{Z}'\boldsymbol{\gamma} + \text{vec}(\mathbf{Z} \otimes \mathbf{Z})'\boldsymbol{\theta} + \varepsilon \quad (2)$$

其中 $\mathbf{Z}$ 为 5 个关键连续变量（ dist_center 、 Lat 、 Lng 、 ladderRatio 、 DOM ）组成的向量。经度和纬度参与扩展以捕捉空间曲面的弯曲，梯户比和挂牌天数参与扩展以允许边际效应递减。 trade_year 不再参与多项式扩展，因为时间固定效应已通过年度虚拟变量灵活捕捉了非线性的时间趋势。 $\mathbf{Z} \otimes \mathbf{Z}$ 包含所有二次项和两两交互项。

该设定允许关键变量存在非线性边际效应，但代价是引入严重的多重共线性（见第五章）。上述线性基准模型（带有时间-区域双向固定效应）是本文进行经济推断的主要

依据，而多项式模型主要用于对比和展示非线性扩展的代价。

4.2 变量选择的共识策略

为在高维特征空间（64 个特征，含时间固定效应）中识别最重要的预测变量，本研究综合运用三种方法：基于 AIC 的向前选择、基于 AIC 的向后剔除，以及 LASSO 正则化路径。采用共识策略确定最终变量集，以增强变量选择的稳健性。

变量选择结果如下：全部 64 个特征中，29 个被三种方法一致选中，30 个被两种方法选中，4 个仅被一种方法选中，1 个未被任何方法选中。最终共识变量集包含 59 个变量。三种方法的一致性程度较高。被一致选中的重要变量包括经纬度、距市中心距离、梯户比、有无电梯、临近地铁、建筑面积对数及大部分区域虚拟变量。年份虚拟变量和多数月份虚拟变量同样被全票选中，验证了时间固定效应纳入模型的必要性。

4.2.1 LASSO 正则化与惩罚力度问题

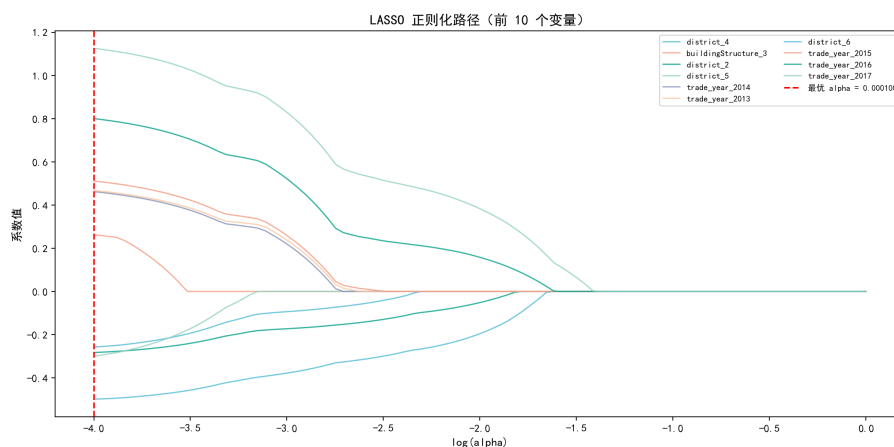


图 6: LASSO 正则化路径

LASSO 交叉验证选择的最优惩罚参数为 $\alpha = 0.0001$ ，恰好位于搜索网格 $[10^{-4}, 10^0]$ 的下界。图 6 显示，模型在此参数下保留了 60/64 个非零系数，惩罚力度极为微弱。将搜索网格向下扩展至 10^{-5} 后，最优 α 仍然落在新的下界，表明这一结果并非网格设定问题所导致。

这一现象的数据解释是：64 个候选特征中绝大多数都与房价存在统计上显著的关联，强预测信号在特征空间中广泛分布，LASSO 无法通过 L1 惩罚将有效信号压缩为零。这与真实模型稀疏的隐含前提相悖，住房价格通常受众多微效因素共同影响。因此 LASSO 在本应用中本质上退化为了普通 OLS，这也解释了为何在后文的模型比较中二者的预测表现几乎完全一致。

5. 模型比较与结果解释

本研究比较了六种回归模型在测试集上的表现：OLS 全变量模型、Ridge 回归、LASSO 回归、OLS 共识筛选变量模型、OLS 与交互项组合以及 OLS 与多项式组合。其中 *trade_year* 因转为虚拟变量不再参与多项式扩展。

5.1 预测表现比较

各模型在测试集上的表现汇总于表 3。

表 3: 测试集模型表现比较

模型	R^2	RMSE (元/m ²)	MAE (元/m ²)	AIC
OLS + 多项式	0.8294	9 411	6 471	−207 770
OLS + 交互项	0.8218	9 621	6 631	−205 038
OLS (全变量)	0.8185	9 696	6 703	−203 859
Ridge (CV)	0.8185	9 696	6 703	—
OLS (筛选变量)	0.8184	9 695	6 703	−203 867
LASSO (CV)	0.8180	9 710	6 713	—

OLS 与多项式组合模型在预测指标上表现最佳， $R^2 = 0.829$ ，RMSE 为 9 411 元/m²，较基准 OLS 提升约 1.1 个百分点。五折交叉验证结果与测试集一致，排除了过拟合的可能。

基准 OLS 模型的 R^2 为 0.818，RMSE 为 9 696 元/m²，表明时间固定效应设定能够有效捕捉房价的非线性时间趋势。OLS 与多项式模型之间的 R^2 差距为 1.1 个百分点，这一差距相对较小，说明时间固定效应已经捕捉了部分原本需要多项式项才能拟合的非线性特征。

5.2 核心变量的边际效应解释

在评估模型时，本研究不仅关注预测精度，更重视核心变量边际效应的经济学含义。以下基于共识变量 OLS 模型对关键变量进行半弹性解释。在对数-线性模型中，连续变量的系数 β 的精确解释为：该变量增加一个单位时，响应变量变化 $100 \times (e^\beta - 1)\%$ 。当系数绝对值较小时， $\beta \times 100\%$ 可作为近似。

首先考察空间因素的影响。距市中心距离的系数为 −2.681，意味着距天安门每增加 0.01 度，房价下降约 2.65%。以 2017 年北京二手房均价约 6 万元/m² 计算，每远离市中心 1 公里，单价下降约 1 590 元。这一效应在统计和经济意义上均高度显著，支持了北京作为单中心城市的空间结构特征。

房屋建筑特征方面，有无电梯的系数为 0.074，有电梯的房源比无电梯房源价格高出约 7.7%。对于均价 6 万元/m² 的住宅，这一溢价约为 4 620 元/m²，折合到一套 80m²

的住房约为 37 万元。梯户比的系数为 0.167，每增加 0.1，房价上升约 1.68%，反映了居住舒适度对房价的资本化效应。

交通便利性方面，临近地铁的系数为 0.056，临近地铁的房源溢价约为 5.8%，体现了轨道交通对住房价值的资本化效应。这一效应在远郊区域可能更为显著，因为远郊居民对轨道交通的依赖度高于中心城区。

图 7 展示了共识变量 OLS 模型中绝对值最大的 15 个系数。可以直观看出，距市中心距离（*dist_center*）的负向效应最为突出，经纬度和区域虚拟变量次之，表明空间因素是房价差异的核心驱动力。房屋属性中，电梯（*elevator*）和梯户比（*ladderRatio*）的正向效应相对显著。

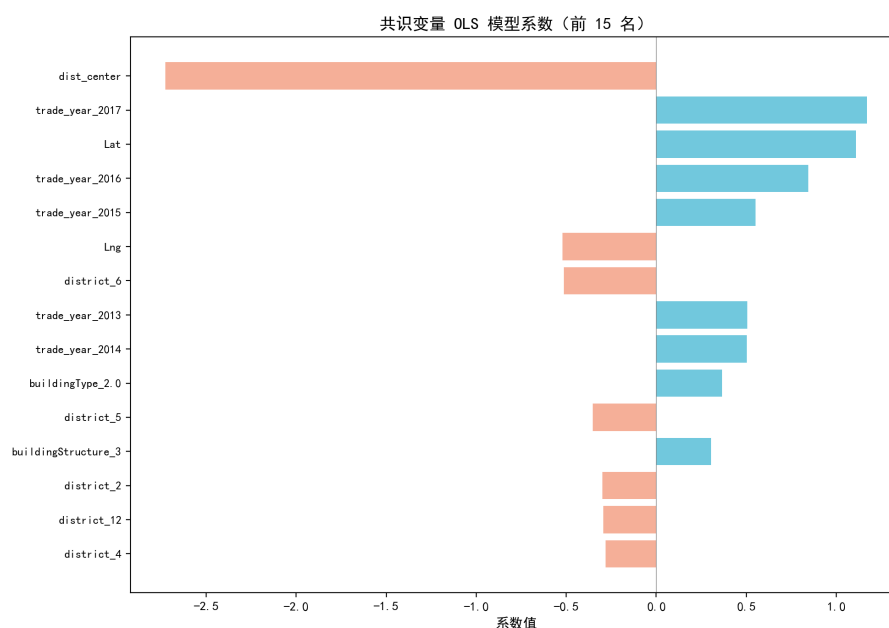


图 7: 共识变量 OLS 模型系数图（前 15 名）

图 8 为测试集上预测值与实际值的散点图。散点沿 45 度参考线紧密分布， $R^2 = 0.8184$ ，表明模型整体拟合效果良好。但在高价区间（ $\log(\text{Price}) > 11.5$ ），散点略向参考线下方偏离，说明模型对极高端房产的预测略有低估。

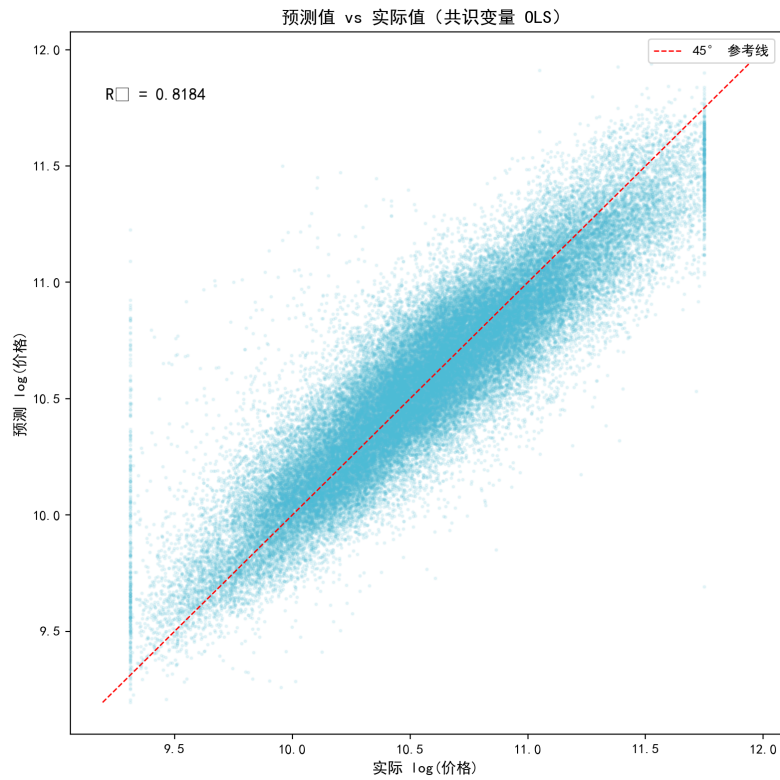


图 8: 预测值 vs 实际值散点图 (共识变量 OLS, 测试集)

图 9 展示了 15 个区域的固定效应系数 (以 *district_1* 为基期)。系数正值越大表示该区域相对于基期的溢价越高。东城区和西城区的系数最高，与中心城区房价领先的现实一致；房山区、平谷区等远郊区系数为负，反映了区位优势的价格折价。

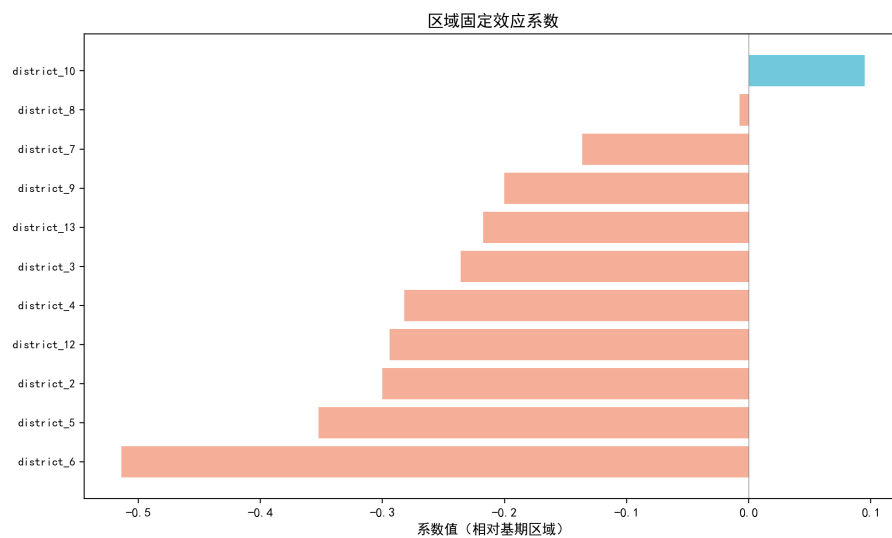


图 9: 区域固定效应系数

5.3 多项式模型的系数膨胀问题

多项式扩展将 5 个关键变量 (*dist_center*、*Lat*、*Lng*、*ladderRatio*、*DOM*) 扩展为 20 个二次项和交互项，部分系数绝对值达到 200 至 800 的量级，在经济意义上不再具有合理的解释力。条件数对比进一步证实了这一问题：全变量 OLS 模型的条件数为 1.67×10^4 ，交互项模型升至 1.41×10^{16} ，多项式模型更高达 1.11×10^{12} ，远超严重多重共线性的经验阈值 (10^6)。

这一现象的根源在于多重共线性。区域虚拟变量 (*district_**) 本身就是地理空间的离散化表示，而经纬度的多项式组合是对同一空间的连续函数逼近。二者高度相关导致正规方程 ($\mathbf{X}'\mathbf{X}$) 近乎奇异，条件数急剧恶化，回归系数在理论上趋向无穷大，呈现出 ± 200 至 ± 800 的反常值。虽然 *trade_year* 已转为虚拟变量不再参与多项式扩展，但 *dist_center* 与 *Lat/Lng* 之间的空间共线性仍然足以引发矩阵病态。

这一发现揭示了一个重要的方法论教训：线性回归模型在引入高阶多项式项后，虽然预测精度有所提升，但其核心优势，即系数可直接解释为边际效应，受到了根本性损害。当研究目标是经济推断 (inference) 而非预测 (prediction) 时，简洁的主效应线性模型在设计合理的固定效应设定下，通常是更为可靠的选择。

5.4 时间趋势的进一步讨论

在共识变量 OLS 模型中，以 2011 年为基期，年份虚拟变量的系数呈现清晰的递增模式：*trade_year_2012* (+0.15)、*trade_year_2013* (+0.48)、*trade_year_2014* (+0.49)、*trade_year_2015* (+0.54)、*trade_year_2016* (+0.83) 和 *trade_year_2017* (+1.15)。这一序列的特征在于：2013 年较 2012 年有大幅跳跃 (+0.33)，而 2013–2015 年间增幅相对平缓，年增约 0.03 至 0.05，2015 年后再次加速。该模式与描述统计分析中观察到的「2011–2015 年初平缓上涨、2015 下半年后加速攀升」完全吻合。

若将 *trade_year* 作为连续线性变量 (系数 0.180)，等效于为所有年份施加了「年增幅约 19.7%」的单一系数，这必然低估 2015 年后的涨幅并高估 2011–2014 年的涨幅。时间固定效应通过年份虚拟变量解决了这一问题，避免了依赖 *trade_year*² 项来部分捕捉非线性趋势的妥协。月度虚拟变量的系数也从 2 月 (+0.046) 逐步递增至 12 月 (+0.190)，反映了一年内的季节性价格上升趋势。

6. 模型诊断

本章的模型诊断基于全变量 OLS 模型的子样本 (20 000 行) 进行。选择子样本的主要原因如下：全量训练集 (254 528 行 $\times$ 64 个特征) 上 *statsmodels* 的影响诊断 (帽子矩阵、Cook's 距离等) 计算开销极大，本团队现有计算设备无法在合理时间内完成全样本的矩阵分解与杠杆值计算。经验表明，20 000 行的样本/特征比已超过 300:1，对于 VIF、Breusch-Pagan 检验、Durbin-Watson 检验等诊断统计量而言，这一样本量在统计推断层面完全充分。后文报告的各项诊断结果均基于该子样本。

6.1 诊断检验结果

表 4: 模型诊断结果汇总

检验项目	结果
VIF > 10 变量数	9 个: buildingStructure 系列、square、log_square、trade_year_2016/2015/2017
最大 VIF (不含常数项)	492.4 (buildingStructure_6)
条件数	4.59×10^5 (较原模型 7.84×10^6 大幅改善)
Breusch-Pagan 检验	LM=957.6, $p = 1.8 \times 10^{-159}$, 拒绝同方差
Goldfeld-Quandt 检验	$F = 0.979$, $p = 0.857$, 不拒绝同方差
Durbin-Watson 统计量	1.966, 无自相关
Cook's D 异常点比例	4.74% (947/20 000) 超过阈值
HC3 下显著性改变量	1 个变量 (fiveYearsProperty_1.0)

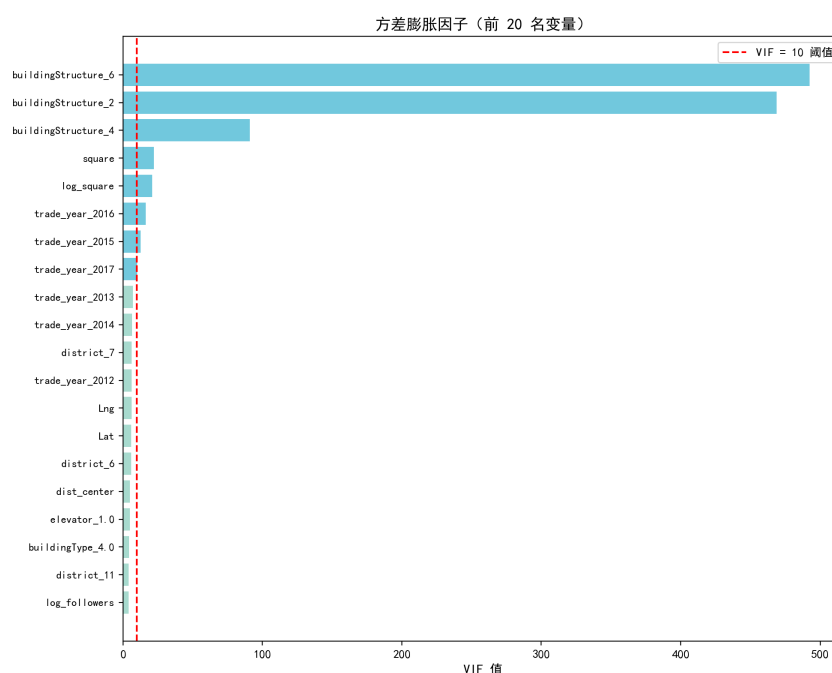


图 10: 方差膨胀因子 (前 20 名变量, 红色虚线为 VIF=10 阈值)

6.2 多重共线性讨论

表 4 报告了 VIF 诊断结果。*buildingStructure_6* 和 *buildingStructure_2* 存在严重的多重共线性。这与 *buildingStructure* 变量的类别分布密切相关: 某些结构类别的占比极低, 导致相应的虚拟变量与其他结构类别之间呈现近乎完全的线性关系。此外, *square* 和 *log_square* 之间的高度相关性也反映了面积变量在对数-线性变换后的共线性问题。

部分年份虚拟变量的 VIF 超过 10，这是以 2011 年为高基期、多年度同时纳入模型时的自然结果。

对于以 OLS 为基础的统计论文，9 个 VIF 大于 10 的变量虽然不影响模型的整体预测能力，但会膨胀个体系数的标准误、降低估计精度。加入时间固定效应后条件数从 7.84×10^6 降至 4.59×10^5 ，这是因为原模型中的 *trade_year* 连续变量与其平方项之间存在大量共线性，被年份虚拟变量替代后这一来源得到消除。在第五节的模型比较中已揭示，多项式模型因高阶项引发的多重共线性严重损害了系数可解释性（条件数升至 1.11×10^{12} ），进一步印证了线性主效应设定的合理性。

6.3 异方差与残差分析

Breusch-Pagan 检验在 1% 水平上显著拒绝同方差假设，而 Goldfeld-Quandt 检验未能检测到异方差。两个检验结论的不一致根源在于检验功效和敏感性的差异：BP 检验对一般形式的异方差均敏感，而 GQ 检验仅对排序后单调变化的方差敏感。

残差诊断图显示，当拟合值增大时残差幅度呈现扩散趋势，这一漏斗状模式支持 BP 检验的结论，即样本中存在不可忽略的异方差。

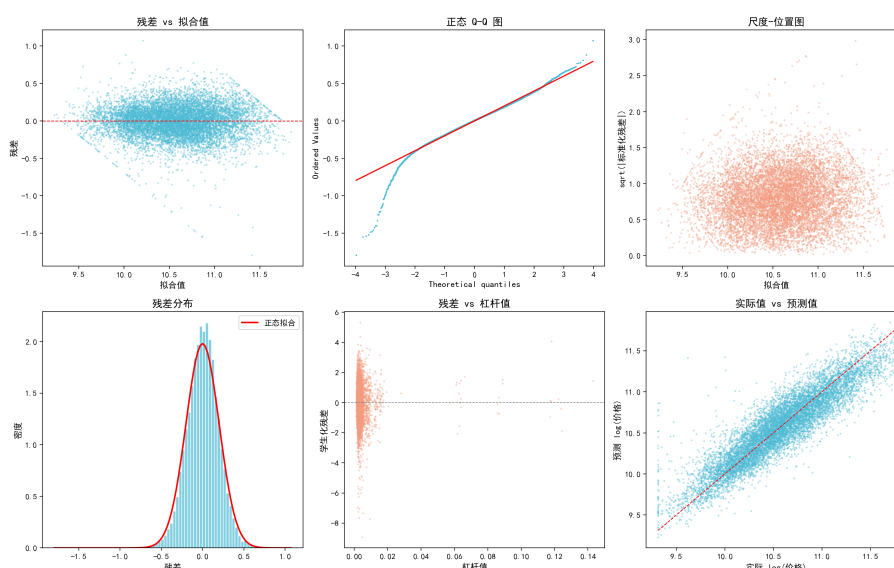


图 11: 残差诊断图（第一行：残差 vs 拟合值、正态 Q-Q 图、尺度-位置图；第二行：残差分布、残差 vs 杠杆值、实际值 vs 预测值）

6.4 稳健标准误

采用 HC3 稳健标准误对 OLS 系数进行校正。结果表明校正后仅一个变量 *five YearsProperty_1.0* 的显著性判断发生改变，从 $p = 0.048$ 变为 $p = 0.053$ ，略超 5% 阈值。这一结果优于原模型，说明加入时间固定效应后异方差对推断的干扰进一步减弱。因此，HC3 稳健标准误的使用主要作为学术严谨性的必要组成部分，而非因为实质性改变了结论。

截面数据的异方差在真实世界中是常态而非异常。本研究从事前和事后两个层面确保了推断的可靠性。事前，半对数设定将绝对差异转化为相对差异，从数据变换的根源上缓解了方差随价格水平递增的趋势。事后，HC3 稳健协方差矩阵的修正进一步消除了异方差对标准误估计的干扰。因此，即便 Breusch-Pagan 检验明确拒绝了同方差假设，核心变量边际效应的推断结果仍然是稳健且可靠的。

6.5 DOM 缺失值处理的稳健性

DOM 缺失率达 50%，为检验其处理方式对核心结论的影响，本文比较了两种设定：包含 DOM 与 *DOM_missing*（基准）和完全剔除 DOM 及其指示变量。结果表明，剔除 DOM 后模型 R^2 从 0.8185 微降至 0.8175，核心变量系数的最大变化不足 1%：距市中心距离系数从 -2.689 变为 -2.698 （差异 0.33%），梯户比系数从 0.168 变为 0.167（差异 0.42%）。此外，*DOM_missing* 的系数显著为负（ -0.028 , $p < 0.001$ ），表明 DOM 缺失的房源系统性地价格偏低，与非随机缺失的推论一致。上述结果证实，DOM 缺失值的处理方式未对核心变量的经济推断构成实质性威胁。

7. 结论

本研究在特征价格模型框架下，基于链家北京 2011–2017 年二手房交易数据，系统比较了六种回归模型的表现。主要发现如下。

空间因素是房价差异的主导变量。距市中心每增加 1 公里，房价下降约 2.7%，呈现显著的单中心城市空间衰减格局。时间因素具有非线性影响，年份虚拟变量系数从 2012 年的 $+0.15$ 递增至 2017 年的 $+1.15$ ，验证了 2015 年前后的结构性加速。房屋特征方面，电梯（ $+7.7\%$ ）、地铁（ $+5.8\%$ ）、梯户比等边际效应在统计和经济意义上均高度显著。

在模型比较层面，多项式模型预测精度最高（ $R^2 = 0.829$ ），但高阶项导致系数膨胀至无法经济解释的程度。纳入时间和区域双向固定效应的线性主效应模型（ $R^2 = 0.818$ ）在可解释性与精度之间取得了更好的平衡。LASSO 因多数特征与房价存在真实关联而退化为 OLS。HC3 稳健标准误的采用仅改变了一个变量的显著性判断，推断结果整体稳健。

本研究存在若干局限。数据仅来源于链家平台，可能存在选择偏误。DOM 变量 50% 的缺失率可能引入填充偏倚。模型未纳入学区等遗漏变量，且未处理空间自相关和内生性问题，结果应解释为条件相关性而非因果关系。关于空间自相关，若邻近房源的误差项存在空间聚类，OLS 估计量仍保持一致性，但标准误可能偏小，导致显著性检验过于宽松。然而，本文核心变量的系数方向和经济显著性均很强（如距市中心距离每公里约 -2.7% ），空间依赖不太可能逆转主要结论的方向。后续研究可考虑空间计量模型、工具变量法及 *buildingStructure* 类别合并等改进方向。

参考文献

- [1] Rosen S. Hedonic prices and implicit markets: product differentiation in pure competition[J]. Journal of Political Economy, 1974, 82(1): 34–55.
- [2] Lancaster K J. A new approach to consumer theory[J]. Journal of Political Economy, 1966, 74(2): 132–157.
- [3] Tibshirani R. Regression shrinkage and selection via the lasso[J]. Journal of the Royal Statistical Society: Series B, 1996, 58(1): 267–288.
- [4] Breusch T S, Pagan A R. A simple test for heteroscedasticity and random coefficient variation[J]. Econometrica, 1979, 47(5): 1287–1294.
- [5] White H. A heteroskedasticity-consistent covariance matrix estimator and a direct test for heteroskedasticity[J]. Econometrica, 1980, 48(4): 817–838.
- [6] Sirmans S G, Macpherson D A, Zietz E N. The composition of hedonic pricing models[J]. Journal of Real Estate Literature, 2005, 13(1): 1–44.
- [7] 郑思齐, 刘洪玉. 住房需求的收入弹性: 模型、估计与预测 [J]. 土木工程学报, 2005, 38(4): 85–89.

附录

本研究的全部数据分析与模型计算均使用 Python 语言完成，主要依赖 pandas、numpy、scipy、matplotlib、seaborn、statsmodels 和 scikit-learn 等开源库。数据来源于 Kaggle 开放数据平台上的链家北京二手房交易数据集 (<https://www.kaggle.com/datasets/ruiqurm/lianjia>)。以下按分析流程列出各脚本的完整代码。

01_exploratory.py ——探索性数据分析

```
import sys
import os
sys.path.insert(0, os.path.dirname(os.path.dirname(os.path.abspath(__file__))))
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from scipy.stats import probplot
from utils import (
    setup_font, load_data, print_section, save_fig, save_table,
    CB_PALETTE, DIVERGING_CMAP, SEQUENTIAL_CMAP,
)
setup_font()
def safe_numeric(series):
    return pd.to_numeric(series, errors="coerce")
def clean_for_eda(df):
    df = df.copy()
    for col in ["livingRoom", "drawingRoom", "bathRoom", "constructionTime"]:
        if col in df.columns and df[col].dtype == object:
            df[col] = safe_numeric(df[col])
    if "DOM" in df.columns:
        df["DOM"] = safe_numeric(df["DOM"])
    df = df[df["price"] > 100].copy()
    return df
print_section("1. 加载数据")
raw = load_data()
df = clean_for_eda(raw)
print(f"价格过滤 (>100) 后: {len(df):,} 行 (剔除 {len(raw) - len(df)} 条)")
print(f"内存占用: {df.memory_usage(deep=True).sum() / 1024 / 1024:.1f} MB")
print_section("2. 基本信息")
print(f"形状: {df.shape}")
print(f"\n列类型: ")
print(df.dtypes.to_string())
missing = df.isnull().sum()
missing_pct = missing / len(df) * 100
missing_info = pd.DataFrame({"缺失数": missing, "占比(%)": missing_pct})
print(f"\n缺失值: ")
print(missing_info[missing_info["缺失数"] > 0].to_string())
```

```

print_section("3. 描述性统计")
desc = df.describe(include="all").T
print(desc.to_string())
save_table(desc.reset_index().rename(columns={"index": "Variable"}), "descriptive_stats.csv")
print_section("4. 目标变量：每平方米价格")
log_price = np.log(df["price"])
fig, axes = plt.subplots(2, 2, figsize=(14, 10))
axes[0, 0].hist(df["price"], bins=80, color=CB_PALETTE[0], edgecolor="white", alpha=0.8)
axes[0, 0].set_xlabel("单价 (元/m²)")
axes[0, 0].set_ylabel("频数")
axes[0, 0].set_title("单价分布")
axes[0, 1].boxplot(df["price"], vert=False, patch_artist=True,
                   boxprops=dict(facecolor=CB_PALETTE[0], alpha=0.6))
axes[0, 1].set_xlabel("单价 (元/m²)")
axes[0, 1].set_title("单价箱线图")
axes[1, 0].hist(log_price, bins=80, color=CB_PALETTE[1], edgecolor="white", alpha=0.8)
axes[1, 0].set_xlabel("log(单价)")
axes[1, 0].set_ylabel("频数")
axes[1, 0].set_title("对数价格分布")
probplot(log_price, dist="norm", plot=axes[1, 1])
axes[1, 1].set_title("对数价格 Q-Q 图")
axes[1, 1].get_lines()[0].set_markerfacecolor(CB_PALETTE[0])
axes[1, 1].get_lines()[0].set_markersize(2)
save_fig("price_distribution.png", "01_eda")
print(f"价格均值: {df['price'].mean():.1f}, 中位数: {df['price'].median():.1f}, "
      f"偏度: {df['price'].skew():.2f}, 范围: [{df['price'].min()}, {df['price'].max()}]")
print(f"对数价格偏度: {log_price.skew():.2f}")
print_section("5. 相关分析")
numeric_cols = [
    "price", "square", "livingRoom", "drawingRoom", "bathRoom",
    "constructionTime", "ladderRatio", "DOM", "followers",
    "Lng", "Lat", "communityAverage", "totalPrice"
]
numeric_cols = [c for c in numeric_cols if c in df.columns]
corr = df[numeric_cols].corr()
mask = np.triu(np.ones_like(corr, dtype=bool), k=1)
fig, ax = plt.subplots(figsize=(12, 10))
sns.heatmap(corr, mask=mask, annot=True, fmt=".2f", cmap=DIVERGING_CMAP,
            center=0, square=True, linewidths=0.5, vmin=-1, vmax=1,
            cbar_kws={"shrink": 0.8})
ax.set_title("数值变量 Pearson 相关系数矩阵")
save_fig("correlation_heatmap.png", "01_eda")
price_corr = corr["price"].drop("price", errors="ignore").sort_values(ascending=False)
print("与价格的相关性 (降序): ")
for var, val in price_corr.items():
    print(f" {var:20s}: {val:.4f}")
print_section("6. 房价地理分布")
fig, ax = plt.subplots(figsize=(12, 10))
geo_sample = df.sample(min(10000, len(df)), random_state=42)

```

```

sc = ax.scatter(geo_sample["Lng"], geo_sample["Lat"],
                c=geo_sample["price"], cmap=SEQUENTIAL_CMAP,
                alpha=0.5, s=5, vmin=0, vmax=np.percentile(df["price"], 98))
plt.colorbar(sc, ax=ax, label="单价（元/㎡）")
ax.set_xlabel("经度")
ax.set_ylabel("纬度")
ax.set_title("北京房价地理分布（抽样 1 万条）")
save_fig("geo_price_scatter.png", "01_eda")
print_section("7. 各区域价格对比")
dist_mean = df.groupby("district")["price"].agg(["mean", "median", "count", "std"])
dist_mean = dist_mean.sort_values("mean", ascending=False)
print(dist_mean.to_string())
fig, ax = plt.subplots(figsize=(12, 5))
ax.bar(range(len(dist_mean)), dist_mean["mean"], color=CB_PALETTE[0], alpha=0.8)
ax.set_xticks(range(len(dist_mean)))
ax.set_xticklabels(dist_mean.index, rotation=45, ha="right")
ax.set_ylabel("平均单价（元/㎡）")
ax.set_title("各区域平均单价")
for i, (idx, row) in enumerate(dist_mean.iterrows()):
    ax.text(i, row["mean"] + 500, f"n={int(row['count']):,}", ha="center", fontsize=7)
save_fig("price_by_district.png", "01_eda")
print_section("8. 价格时间趋势")
df["trade_year"] = pd.to_datetime(df["tradeTime"], errors="coerce").dt.year
df = df[df["trade_year"].between(2011, 2017)].copy()
print(f"过滤交易年份后: {len(df):,} 行 (2011-2017)")
yearly = df.groupby("trade_year")["price"].agg(["mean", "median", "count", "std"])
yearly = yearly.dropna()
print(yearly.to_string())
fig, ax1 = plt.subplots(figsize=(10, 5))
ax1.plot(yearly.index, yearly["mean"], "o-", color=CB_PALETTE[0], label="平均价格", linewidth=2)
ax1.plot(yearly.index, yearly["median"], "s--", color=CB_PALETTE[1], label="中位数价格", linewidth=2)
ax1.fill_between(yearly.index,
                 yearly["mean"] - yearly["std"],
                 yearly["mean"] + yearly["std"],
                 alpha=0.15, color=CB_PALETTE[0], label="±1 标准差")
ax1.set_xlabel("年份")
ax1.set_ylabel("单价（元/㎡）")
ax1.set_title("北京房价走势（2011-2017）")
ax1.legend(loc="upper left")
ax2 = ax1.twinx()
ax2.bar(yearly.index, yearly["count"], alpha=0.15, color=CB_PALETTE[2], label="交易量")
ax2.set_ylabel("交易量")
lines1, labels1 = ax1.get_legend_handles_labels()
lines2, labels2 = ax2.get_legend_handles_labels()
ax1.legend(lines1 + lines2, labels1 + labels2, loc="upper left")
save_fig("price_trend_by_year.png", "01_eda")
print_section("9. 分类变量对比")
cat_vars = [

```

```

("subway", "是否近地铁"),
("elevator", "是否有电梯"),
("fiveYearsProperty", "是否满五年"),
("buildingType", "建筑类型"),
("renovationCondition", "装修情况"),
("buildingStructure", "建筑结构"),
]
for col, label in cat_vars:
    if col not in df.columns or df[col].nunique() > 15:
        continue
    fig, ax = plt.subplots(figsize=(8, 4))
    df.boxplot(column="price", by=col, ax=ax, grid=False, patch_artist=True)
    ax.set_title(f"不同 {label} 的房价分布")
    ax.set_ylabel("单价 (元/m²)")
    ax.set_xlabel(label)
    fig.suptitle("")
    save_fig(f"price_by_{col}.png", "01_eda")
print_section("10. 数值变量分布")
num_plot_cols = ["square", "DOM", "followers", "constructionTime", "ladderRatio"]
fig, axes = plt.subplots(2, 3, figsize=(15, 10))
axes = axes.flatten()
for i, col in enumerate(num_plot_cols):
    if col not in df.columns:
        continue
    if col == "DOM":
        valid = df[col].dropna()
        valid = valid[valid < valid.quantile(0.99)]
    elif col == "followers":
        valid = df[col]
        valid = valid[valid < valid.quantile(0.99)]
    elif col == "ladderRatio":
        valid = df[col].dropna()
        valid = valid[valid < valid.quantile(0.99)]
    elif col == "constructionTime":
        valid = df[col].dropna()
    else:
        valid = df[col].dropna()
    axes[i].hist(valid, bins=60, color=CB_PALETTE[0], edgecolor="white", alpha=0.8)
    axes[i].set_xlabel(col)
    axes[i].set_ylabel("频数")
    axes[i].set_title(f"{col} 的分布")
axes[-1].set_visible(False)
save_fig("numeric_distributions.png", "01_eda")
print_section("11. 关键变量散点图矩阵 (抽样)")
pair_cols = ["price", "square", "constructionTime", "Lng", "Lat"]
pair_df = df[pair_cols].dropna().sample(min(5000, len(df)), random_state=42)
pair_df["log_price"] = np.log(pair_df["price"])
sns.pairplot(pair_df.drop(columns=["price"]), diag_kind="kde",
              plot_kws={"alpha": 0.3, "s": 5})

```

```

save_fig("pairplot_key_vars.png", "01_eda")
print_section("12. 关键数值结果")
print(f"""
数据集: {len(df):,} 条交易, {int(df['trade_year'].min())}-{int(df['trade_year'].max())} 年, {df['district'].nunique()} 个区域
价格均值={df['price'].mean():.1f}, 中位数={df['price'].median():.1f}, 偏度={df['price'].skew():.2f} (对数后: {log_price.skew():.2f})
与价格相关性最高: communityAverage={price_corr.get('communityAverage', 'N/A'):.4f}, totalPrice={price_corr.get('totalPrice', 'N/A'):.4f}

```

02_preprocessing.py ——数据预处理

```

import sys
import os
sys.path.insert(0, os.path.dirname(os.path.dirname(os.path.abspath(__file__))))
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from scipy.stats.mstats import winsorize
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
import joblib
from utils import (
    setup_font, load_data, print_section, save_fig, save_table, CB_PALETTE
)
setup_font()
SEED = 42
print_section("1. 加载原始数据")
df = load_data()
print(f"原始形状: {df.shape}")
print_section("2. 删除有问题的列")
drop_cols = ["communityAverage", "totalPrice", "Cid", "url", "id"]
df.drop(columns=drop_cols, inplace=True)
print(f"已删除: {drop_cols}")
print(f"删除后形状: {df.shape}")
print_section("3. 修正数据类型")
for col in ["livingRoom", "drawingRoom", "bathRoom", "constructionTime", "DOM"]:
    df[col] = pd.to_numeric(df[col], errors="coerce")
    print(f" {col}: {df[col].dtype}, {df[col].isna().sum()} 个被强制转为 NaN")
print_section("4. 过滤极端值")
n_before = len(df)
df = df[df["price"] > 100].copy()
df = df[(df["constructionTime"].isna()) | (df["constructionTime"] >= 1920) &
        (df["constructionTime"] <= 2017)]
df = df[(df["bathRoom"].isna()) | (df["bathRoom"] <= 10)]
df = df[(df["livingRoom"].isna()) | (df["livingRoom"] <= 15)]
df = df[(df["drawingRoom"].isna()) | (df["drawingRoom"] <= 10)]
df = df[(df["square"] >= 10) & (df["square"] <= 1000)]

```

```

n_removed = n_before - len(df)
print(f"删除了 {n_removed} 行极端值, 当前形状: {df.shape}")
print_section("5. 处理缺失值")
missing_before = df.isnull().sum()
print("插补前的缺失情况: ")
print(missing_before[missing_before > 0].to_string())
df["DOM_missing"] = df["DOM"].isna().astype(int)
df["DOM"].fillna(df["DOM"].median(), inplace=True)
df["constructionTime"].fillna(df["constructionTime"].median(), inplace=True)
cat_impute = ["buildingType", "elevator", "fiveYearsProperty", "subway"]
for col in cat_impute:
    df[col].fillna(df[col].mode()[0], inplace=True)
for col in ["livingRoom", "drawingRoom", "bathRoom"]:
    df[col].fillna(df[col].median(), inplace=True)
missing_after = df.isnull().sum()
print("\n插补后的缺失情况: ")
print(missing_after[missing_after > 0].to_string() if missing_after.sum() > 0 else " 无缺失! ")
print_section("6. 异常值处理 (Winsorize 截尾)")
winsorize_cols = {
    "price": (0.005, 0.005),
    "square": (0.005, 0.005),
    "ladderRatio": (0.01, 0.01),
    "followers": (0.01, 0.01),
    "DOM": (0.01, 0.01),
}
for col, (lower, upper) in winsorize_cols.items():
    original_std = df[col].std()
    df[col] = winsorize(df[col], limits=(lower, upper))
    new_std = df[col].std()
    print(f" {col}: 标准差 {original_std:.1f} -> {new_std:.1f}")
print_section("7. 解析楼层变量")
def parse_floor(s):
    s = str(s)
    parts = s.split()
    if len(parts) == 0:
        return None, None
    level = parts[0]
    num = parts[1] if len(parts) > 1 else None
    level_map = {"低": "low", "中": "mid", "高": "high", "地": "ground"}
    level = level_map.get(level, level)
    return level, num
floor_data = df["floor"].apply(parse_floor)
df["floor_level"] = floor_data.apply(lambda x: x[0])
df["floor_num"] = pd.to_numeric(floor_data.apply(lambda x: x[1]), errors="coerce")
df["floor_num"].fillna(df["floor_num"].median(), inplace=True)
print(f" 楼层类别分布: \n{df['floor_level'].value_counts().to_string()}")
print_section("8. 创建衍生变量")
df["tradeTime"] = pd.to_datetime(df["tradeTime"], errors="coerce")
df["trade_year"] = df["tradeTime"].dt.year

```

```

df["trade_month"] = df["tradeTime"].dt.month
df["property_age"] = df["trade_year"] - df["constructionTime"]
df["property_age"] = df["property_age"].clip(lower=0, upper=100)
beijing_center = (116.3974, 39.9082)
df["dist_center"] = np.sqrt(
    (df["Lng"] - beijing_center[0]) ** 2 +
    (df["Lat"] - beijing_center[1]) ** 2
)
df["log_price"] = np.log(df["price"])
df["log_square"] = np.log(df["square"])
df["log_followers"] = np.log1p(df["followers"])
n_before_year = len(df)
df = df[df["trade_year"].between(2011, 2017)].copy()
print(f" 过滤交易年份: {n_before_year - len(df)} 行被删除 (不在 2011-2017 范围内)")
print(" 已创建: trade_year, trade_month, property_age, dist_center, log_price, log_square,
      log_followers")
print_section("9. 编码分类变量")
cat_cols = {
    "buildingType": "category",
    "buildingStructure": "category",
    "renovationCondition": "category",
    "elevator": "category",
    "subway": "category",
    "fiveYearsProperty": "category",
    "floor_level": "category",
    "district": "category",
    "trade_year": "category",
    "trade_month": "category",
}
for col, dtype in cat_cols.items():
    df[col] = df[col].astype(dtype)
df_encoded = pd.get_dummies(df, columns=list(cat_cols.keys()), drop_first=True, dtype=int)
print(f"编码后形状: {df_encoded.shape}")
print_section("10. 定义特征矩阵和标签")
target = "log_price"
exclude = [
    "log_price", "price", "tradeTime", "constructionTime",
    "floor",
]
feature_cols = [c for c in df_encoded.columns if c not in exclude]
feature_cols = [c for c in feature_cols if df_encoded[c].dtype in [np.float64, np.int64, int, np.
    int32, np.float32]]
X = df_encoded[feature_cols]
y = df_encoded[target]
print(f"特征矩阵: {X.shape}")
print(f"目标变量: {target}")
print(f"特征列表 ({len(feature_cols)} 个): ")
for i, c in enumerate(feature_cols):
    print(f" {i+1:3d}. {c}")

```

```

print_section("11. 训练-测试集划分")
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=SEED
)
print(f"训练集: {X_train.shape[0]:,} 行")
print(f"测试集: {X_test.shape[0]:,} 行")
print_section("12. 标准化（用于正则化模型）")
dummy_patterns = [
    "buildingType_", "buildingStructure_", "renovationCondition_",
    "elevator_", "subway_", "fiveYearsProperty_", "floor_level_", "district_",
    "trade_year_", "trade_month_"
]
dummy_cols = [c for c in feature_cols if any(p in c for p in dummy_patterns)]
numeric_cols = [c for c in feature_cols if c not in dummy_cols]
print(f"数值型（已标准化）: {len(numeric_cols)}")
print(f"虚拟变量（未标准化）: {len(dummy_cols)}")
scaler = StandardScaler()
X_train_scaled = X_train.copy()
X_test_scaled = X_test.copy()
X_train_scaled[numeric_cols] = scaler.fit_transform(X_train[numeric_cols])
X_test_scaled[numeric_cols] = scaler.transform(X_test[numeric_cols])
print("标准化完成。")
print_section("13. 保存处理结果")
joblib.dump((X_train, X_test, y_train, y_test, feature_cols),
            "output/processed_data.pkl")
joblib.dump((X_train_scaled, X_test_scaled, numeric_cols, dummy_cols),
            "output/scaled_data.pkl")
joblib.dump(scaler, "output/scaler.pkl")
print("已保存: ")
print(" output/processed_data.pkl （未标准化的训练/测试集）")
print(" output/scaled_data.pkl （标准化后的训练/测试集 + 列名）")
print(" output/scaler.pkl （StandardScaler 对象）")
print_section("14. 关键数值结果")
print(f"""
最终: {X_train.shape[0]:,} 训练 / {X_test.shape[0]:,} 测试, {len(feature_cols)} 个特征 ({len(
    numeric_cols)} 个数值型 + {len(dummy_cols)} 个虚拟变量)
已删除 {n_removed} 行极端值, 插补 {missing_before.sum()} 个缺失值
Winsorize 截尾: price(0.5%), square(0.5%), ladderRatio(1%), followers(1%), DOM(1%)

```

03_variable_selection.py ——变量选择

```

import sys
import os
sys.path.insert(0, os.path.dirname(os.path.dirname(os.path.abspath(__file__))))
import warnings
warnings.filterwarnings("ignore")
import pandas as pd
import numpy as np

```

```

import matplotlib.pyplot as plt
import joblib
import statsmodels.api as sm
from sklearn.linear_model import LassoCV, Lasso
import logging
logging.getLogger("statsmodels").setLevel(logging.WARNING)
from utils import (
    setup_font, print_section, save_fig, save_table, CB_PALETTE
)
setup_font()
SEED = 42
np.random.seed(SEED)
print_section("1. 加载预处理后的数据")
X_train, X_test, y_train, y_test, feature_cols = joblib.load("output/processed_data.pkl")
X_train_scaled, X_test_scaled, numeric_cols, dummy_cols = joblib.load("output/scaled_data.pkl")
print(f"全量训练集: {X_train.shape[0]:,} 行 x {X_train.shape[1]} 个特征")
print_section("2. 向前选择 (基于 AIC, 子样本)")
subsample_size = min(20000, len(X_train))
idx_sub = np.random.choice(X_train.index, subsample_size, replace=False)
X_sub = X_train.loc[idx_sub]
y_sub = y_train.loc[idx_sub]
print(f"子样本: {subsample_size:,} 行")
def forward_selection_aic(X, y, max_features=30):
    remaining = list(X.columns)
    selected = []
    current_aic = np.inf
    history = []
    for step in range(max_features):
        if not remaining:
            break
        best_var = None
        best_aic = np.inf
        best_model = None
        for var in remaining:
            cols = selected + [var]
            X_subset = sm.add_constant(X[cols])
            try:
                model = sm.OLS(y, X_subset).fit()
                if model.aic < best_aic:
                    best_aic = model.aic
                    best_var = var
                    best_model = model
            except Exception:
                continue
        if best_aic < current_aic - 1e-6:
            selected.append(best_var)
            remaining.remove(best_var)
            current_aic = best_aic
            history.append({"step": step + 1, "var": best_var, "aic": best_aic,

```

```

        "rsquared": best_model.rsquared})
    print(f" 第{step+1:2d}步: + {best_var:35s} AIC={best_aic:.1f}, R2={best_model.rsquared
        :.4f}")
else:
    print(f" 在第 {step} 步停止: AIC 不再改善")
    break

return selected, pd.DataFrame(history)
selected_forward, fwd_history = forward_selection_aic(X_sub, y_sub)
print(f"\n向前选择完毕: {len(selected_forward)} 个变量")
print_section("3. 向后剔除 (基于 AIC, 子样本)")
def backward_elimination_aic(X, y, max_iter=60):
    cols = list(X.columns)
    history = []
    for step in range(max_iter):
        if len(cols) <= 1:
            break
        X_subset = sm.add_constant(X[cols])
        try:
            model = sm.OLS(y, X_subset).fit()
        except Exception:
            break
        current_aic = model.aic
        pvalues = model.pvalues.drop("const", errors="ignore")
        worst_var = pvalues.idxmax()
        worst_pval = pvalues.max()
        cols_reduced = [c for c in cols if c != worst_var]
        X_reduced = sm.add_constant(X[cols_reduced])
        try:
            model_reduced = sm.OLS(y, X_reduced).fit()
            if model_reduced.aic <= current_aic + 1e-6:
                cols = cols_reduced
                history.append({"step": step + 1, "removed": worst_var,
                               "pval": worst_pval, "aic": model_reduced.aic,
                               "rsquared": model_reduced.rsquared})
                print(f" 第{step+1:2d}步: - {worst_var:35s} p={worst_pval:.6f}, AIC={model_reduced.
                    aic:.1f}")
            else:
                print(f" 停止: 剔除 {worst_var} 使 AIC 上升")
                break
        except Exception:
            break

    return cols, pd.DataFrame(history)
selected_backward, bwd_history = backward_elimination_aic(X_sub, y_sub)
print(f"\n向后剔除完毕: {len(selected_backward)} 个变量保留")
print_section("4. LASSO 正则化 (全量数据)")
lasso_cv = LassoCV(
    alphas=np.logspace(-4, 0, 100),
    cv=5,
    max_iter=10000,

```

```

    random_state=SEED,
    n_jobs=-1
)
lasso_cv.fit(X_train_scaled, y_train)
print(f"最优 alpha: {lasso_cv.alpha_:.6f}")
print(f"非零系数: {np.sum(lasso_cv.coef_ != 0)} / {len(feature_cols)}")
alphas = lasso_cv.alphas_
coef_path = []
for alpha in alphas:
    lasso = Lasso(alpha=alpha, max_iter=10000)
    lasso.fit(X_train_scaled, y_train)
    coef_path.append(lasso.coef_)
coef_path = np.array(coef_path)
top10_idx = np.argsort(np.abs(lasso_cv.coef_))[-10:]
top10_names = [feature_cols[i] for i in top10_idx]
fig, ax = plt.subplots(figsize=(12, 6))
for rank, i in enumerate(top10_idx):
    color = CB_PALETTE[rank % len(CB_PALETTE)]
    ax.plot(np.log10(alphas), coef_path[:, i],
            alpha=0.8, linewidth=1.2, color=color,
            label=feature_cols[i])
ax.axvline(np.log10(lasso_cv.alpha_), color="red", linestyle="--",
            label=f"最优 alpha = {lasso_cv.alpha_:.6f}")
ax.set_xlabel("log(alpha)")
ax.set_ylabel("系数值")
ax.set_title("LASSO 正则化路径 (前 10 个变量)")
ax.legend(loc="best", fontsize=8, ncol=2)
save_fig("lasso_path.png", "03_selection")
lasso_coef = pd.Series(lasso_cv.coef_, index=feature_cols)
lasso_selected = list(lasso_coef[abs(lasso_coef) > 1e-4].sort_values(key=abs, ascending=False).
    index)
print(f"\nLASSO 系数绝对值前 15 名: ")
for var, coef in lasso_coef[abs(lasso_coef) > 1e-4].abs().sort_values(ascending=False).head(15).
    items():
    print(f" {var:35s}: {lasso_cv.coef_[feature_cols.index(var)]:+.6f}")
print_section("5. 比较三种选择方法")
var_list = []
for v in feature_cols:
    row = {"Variable": v,
           "Forward": 1 if v in selected_forward else 0,
           "Backward": 1 if v in selected_backward else 0,
           "LASSO": 1 if abs(lasso_cv.coef_[feature_cols.index(v)]) > 1e-4 else 0}
    var_list.append(row)
comparison = pd.DataFrame(var_list)
comparison["Agreement"] = comparison[["Forward", "Backward", "LASSO"]].sum(axis=1)
comparison = comparison.sort_values(["Agreement", "Variable"], ascending=[False, True])
comparison["Selected"] = comparison["Agreement"] >= 2
save_table(comparison, "selection_comparison.csv")
print("三种方法一致性比较 (被至少 2 种方法选中的变量): ")

```

```

selected_by_consensus = comparison[comparison["Agreement"] >= 2]
print(selected_by_consensus[["Variable", "Forward", "Backward", "LASSO", "Agreement"]].to_string(
    ()))
print_section("6. 基于共识变量的 OLS 模型 (一致性 >= 2) ")
consensus_vars = comparison[comparison["Agreement"] >= 2][["Variable"]].tolist()
print(f"共识变量 ({len(consensus_vars)} 个): ")
for v in consensus_vars:
    print(f" - {v}")
if len(consensus_vars) == 0:
    print("警告: 无变量被至少 2 种方法选中, 回退到 LASSO 选中的变量。")
    consensus_vars = lasso_selected[:20]
X_train_consensus = sm.add_constant(X_train[consensus_vars])
ols_consensus = sm.OLS(y_train, X_train_consensus).fit()
print(f"\nOLS R2: {ols_consensus.rsquared:.4f}")
print(f"OLS 调整 R2: {ols_consensus.rsquared_adj:.4f}")
print(f"AIC: {ols_consensus.aic:.1f}")
print(f"\|系数|前十: ")
coef_df = pd.DataFrame({
    "Variable": ols_consensus.params.index,
    "Coefficient": ols_consensus.params.values,
    "P-value": ols_consensus.pvalues.values,
    "Significant": ols_consensus.pvalues.values < 0.05
}).sort_values("Coefficient", key=abs, ascending=False)
print(coef_df.head(10).to_string())
save_table(coef_df, "consensus_ols_coefficients.csv")
print_section("7. 保存选择结果")
selection_results = {
    "selected_forward": selected_forward,
    "selected_backward": selected_backward,
    "lasso_selected": lasso_selected,
    "consensus_vars": consensus_vars,
}
joblib.dump(selection_results, "output/selection_results.pkl")
print("已保存: output/selection_results.pkl")
print_section("8. 关键数值结果")
print(f"""
向前选择: {len(selected_forward)} 个 | 向后剔除: {len(selected_backward)} 个保留 | LASSO: {len(
    lasso_selected)} 个非零系数 (alpha={lasso_cv.alpha_.6f})
共识变量 (2 种方法): {len(consensus_vars)} 个

```

04_model_diagnostics.py ——模型诊断

```

import sys
import os
sys.path.insert(0, os.path.dirname(os.path.dirname(os.path.abspath(__file__))))
import warnings
warnings.filterwarnings("ignore")
import time

```

```

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import joblib
import statsmodels.api as sm
from scipy import stats
from statsmodels.stats.outliers_influence import variance_inflation_factor
from statsmodels.stats.diagnostic import het_breuschpagan, het_goldfeldquandt
from statsmodels.stats.stattools import durbin_watson
import logging
logging.getLogger("statsmodels").setLevel(logging.WARNING)
from sklearn.preprocessing import PolynomialFeatures
from utils import (
    setup_font, print_section, save_fig, save_table, CB_PALETTE
)
setup_font()
SEED = 42
np.random.seed(SEED)
t0 = time.time()
print_section("1. 加载数据")
X_train, X_test, y_train, y_test, feature_cols = joblib.load("output/processed_data.pkl")
print(f"训练集: {X_train.shape[0]:,} 行 x {X_train.shape[1]} 个特征")
print(f"测试集: {X_test.shape[0]:,} 行")
print_section("2. 全变量 OLS 模型")
diag_sample = min(20000, len(X_train))
idx_diag = np.random.choice(X_train.index, diag_sample, replace=False)
X_diag = X_train.loc[idx_diag]
y_diag = y_train.loc[idx_diag]
print(f"在 {diag_sample:,} 行上用 {len(feature_cols)} 个特征拟合 OLS...")
X_diag_const = sm.add_constant(X_diag)
ols_full = sm.OLS(y_diag, X_diag_const).fit()
print(ols_full.summary())
print(f"OLS 拟合用时: {time.time() - t0:.1f}s")
t1 = time.time()
fitted = ols_full.fittedvalues
residuals = ols_full.resid
X_mat = X_diag_const.values.astype(np.float64)
n, p = X_mat.shape
inv_xtx = np.asarray(ols_full.normalized_cov_params)
x_cov = X_mat @ inv_xtx
hat_diag = (x_cov * X_mat).sum(axis=1)
hat_diag = np.clip(hat_diag, 0, 1 - 1e-10)
resid_vals = residuals.values.ravel()
mse_resid = ols_full.mse_resid
resid_student = resid_vals / (np.sqrt(mse_resid) * np.sqrt(1 - hat_diag))
sigma_i = np.sqrt(((n - p) / (n - p - 1)) * mse_resid -
                  resid_vals**2 / ((n - p - 1) * (1 - hat_diag)))
studentized_resid = resid_vals / (sigma_i * np.sqrt(1 - hat_diag))
cooks = studentized_resid**2 / p * hat_diag / (1 - hat_diag)

```

```

leverage = hat_diag
print(f" 影响诊断计算用时: {time.time() - t1:.1f}s")
t1 = time.time()
print_section("3. 残差分析")
plot_n = min(10000, diag_sample)
plot_idx = np.random.choice(diag_sample, plot_n, replace=False)
fig, axes = plt.subplots(2, 3, figsize=(16, 10))
axes[0, 0].scatter(fitted.iloc[plot_idx], residuals.iloc[plot_idx],
                  alpha=0.3, s=2, color=CB_PALETTE[0])
axes[0, 0].axhline(y=0, color="red", linestyle="--", linewidth=1)
axes[0, 0].set_xlabel("拟合值")
axes[0, 0].set_ylabel("残差")
axes[0, 0].set_title("残差 vs 拟合值")
stats.probplot(residuals, dist="norm", plot=axes[0, 1])
axes[0, 1].get_lines()[0].set_markersize(1)
axes[0, 1].get_lines()[0].set_mahoutedgecolor(CB_PALETTE[0])
axes[0, 1].get_lines()[1].set_color("red")
axes[0, 1].set_title("正态 Q-Q 图")
axes[0, 2].scatter(fitted.iloc[plot_idx],
                  np.sqrt(np.abs(studentized_resid[plot_idx])),
                  alpha=0.3, s=2, color=CB_PALETTE[1])
axes[0, 2].set_xlabel("拟合值")
axes[0, 2].set_ylabel("sqrt(|标准化残差|)")
axes[0, 2].set_title("尺度-位置图")
axes[1, 0].hist(residuals, bins=80, density=True, alpha=0.7, color=CB_PALETTE[0], edgecolor="
white")
x_range = np.linspace(residuals.min(), residuals.max(), 100)
axes[1, 0].plot(x_range, stats.norm.pdf(x_range, residuals.mean(), residuals.std()),
               "r-", linewidth=2, label="正态拟合")
axes[1, 0].set_xlabel("残差")
axes[1, 0].set_ylabel("密度")
axes[1, 0].set_title("残差分布")
axes[1, 0].legend()
axes[1, 1].scatter(leverage[plot_idx], studentized_resid[plot_idx],
                  alpha=0.3, s=2, color=CB_PALETTE[1])
axes[1, 1].axhline(y=0, color="gray", linestyle="--", linewidth=0.8)
axes[1, 1].set_xlabel("杠杆值")
axes[1, 1].set_ylabel("学生化残差")
axes[1, 1].set_title("残差 vs 杠杆值")
axes[1, 2].scatter(y_diag.iloc[plot_idx], fitted.iloc[plot_idx],
                  alpha=0.3, s=2, color=CB_PALETTE[0])
axes[1, 2].plot([y_diag.min(), y_diag.max()],
               [y_diag.min(), y_diag.max()], "r--", linewidth=1)
axes[1, 2].set_xlabel("实际 log(价格)")
axes[1, 2].set_ylabel("预测 log(价格)")
axes[1, 2].set_title("实际值 vs 预测值")
save_fig("residual_diagnostics.png", "04_diagnostics")
print(f" 残差图绘图用时: {time.time() - t1:.1f}s")
t1 = time.time()

```

```

print_section("4. 多重共线性 (VIF) ")
vif_sample = min(20000, len(X_train))
idx_vif = np.random.choice(X_train.index, vif_sample, replace=False)
X_vif = sm.add_constant(X_train.loc[idx_vif])
vif_data = []
for i in range(X_vif.shape[1]):
    vif_val = variance_inflation_factor(X_vif.values, i)
    vif_data.append({
        "Variable": X_vif.columns[i],
        "VIF": round(vif_val, 2),
        "VIF > 10": vif_val > 10
    })
vif_df = pd.DataFrame(vif_data).sort_values("VIF", ascending=False)
print(f"VIF 在 {vif_sample:,} 行上计算, 共 {len(vif_df)} 个变量")
print("\nVIF 前 20 名: ")
print(vif_df.head(20).to_string())
print(f"\nVIF > 10 的变量数: {vif_df['VIF > 10'].sum()}")
save_table(vif_df, "vif_results.csv")
vif_no_const = vif_df[vif_df["Variable"] != "const"]
vif_top20 = vif_no_const.head(20).sort_values("VIF", ascending=True)
fig, ax = plt.subplots(figsize=(10, 8))
colors_vif = [CB_PALETTE[0] if v > 10 else CB_PALETTE[3]
               for v in vif_top20["VIF"]]
ax.barh(range(len(vif_top20)), vif_top20["VIF"], color=colors_vif, alpha=0.8)
ax.set_yticks(range(len(vif_top20)))
ax.set_yticklabels(vif_top20["Variable"], fontsize=9)
ax.axvline(x=10, color="red", linestyle="--", linewidth=1.5, label="VIF = 10 阈值")
ax.set_xlabel("VIF 值")
ax.set_title("方差膨胀因子 (前 20 名变量)")
ax.legend()
save_fig("vif_bar_chart.png", "04_diagnostics")
print(f" VIF 用时: {time.time() - t1:.1f}s")
t1 = time.time()
print_section("4b. 多项式模型条件数")
key_interact_vars = ["dist_center", "Lat", "Lng", "ladderRatio", "DOM"]
key_interact_vars = [v for v in key_interact_vars if v in feature_cols]
remaining_vars = [v for v in feature_cols if v not in key_interact_vars]
poly_full = PolynomialFeatures(degree=2, interaction_only=False, include_bias=False)
X_poly = np.hstack([
    X_train[remaining_vars].values,
    poly_full.fit_transform(X_train[key_interact_vars].values)
])
poly_feat_names = remaining_vars + poly_full.get_feature_names_out(key_interact_vars).tolist()
X_poly_df = pd.DataFrame(X_poly, columns=poly_feat_names, index=X_train.index)
poly_sample = min(5000, len(X_poly_df))
idx_poly = np.random.choice(X_poly_df.index, poly_sample, replace=False)
X_poly_sub = X_poly_df.loc[idx_poly].values
X_poly_std = (X_poly_sub - X_poly_sub.mean(axis=0)) / (X_poly_sub.std(axis=0) + 1e-10)
cond_poly = np.linalg.cond(X_poly_std)

```

```

print(f"多项式模型特征数: {X_poly_df.shape[1]}")
print(f"条件数 (5,000行子样本, 标准化后): {cond_poly:.2e}")
poly_interact = PolynomialFeatures(degree=2, interaction_only=True, include_bias=False)
X_interact = np.hstack([
    X_train[remaining_vars].values,
    poly_interact.fit_transform(X_train[key_interact_vars].values)
])
interact_feat_names = remaining_vars + poly_interact.get_feature_names_out(key_interact_vars).
    tolist()
X_interact_df = pd.DataFrame(X_interact, columns=interact_feat_names, index=X_train.index)
idx_int = np.random.choice(X_interact_df.index, poly_sample, replace=False)
X_int_sub = X_interact_df.loc[idx_int].values
X_int_std = (X_int_sub - X_int_sub.mean(axis=0)) / (X_int_sub.std(axis=0) + 1e-10)
cond_int = np.linalg.cond(X_int_std)
print(f"交互项模型特征数: {X_interact_df.shape[1]}")
print(f"条件数 (5,000行子样本, 标准化后): {cond_int:.2e}")
print(f" 多项式条件数用时: {time.time() - t1:.1f}s")
t1 = time.time()
print_section("5. 异方差检验")
bp_test = het_breuschpagan(residuals, X_diag_const)
bp_stat, bp_pval, bp_fstat, bp_fpval = bp_test
print(f"Breusch-Pagan 检验: ")
print(f" LM 统计量: {bp_stat:.4f}")
print(f" LM p 值: {bp_pval:.6e}")
print(f" F 统计量: {bp_fstat:.4f}")
print(f" F p 值: {bp_fpval:.6e}")
gq_test = het_goldfeldquandt(residuals, X_diag_const)
gq_stat, gq_pval, gq_alt = gq_test
print(f"\nGoldfeld-Quandt 检验: ")
print(f" F 统计量: {gq_stat:.4f}")
print(f" p 值: {gq_pval:.6e}")
print(f" 对立假设: {gq_alt}")
het_results = pd.DataFrame({
    "Test": ["Breusch-Pagan (LM)", "Breusch-Pagan (F)", "Goldfeld-Quandt"],
    "Statistic": [bp_stat, bp_fstat, gq_stat],
    "p-value": [bp_pval, bp_fpval, gq_pval],
    "Conclusion": [
        "异方差" if bp_pval < 0.05 else "同方差",
        "异方差" if bp_fpval < 0.05 else "同方差",
        "异方差" if gq_pval < 0.05 else "同方差",
    ]
})
save_table(het_results, "heteroscedasticity_tests.csv")
print(het_results.to_string())
print(f" 异方差检验用时: {time.time() - t1:.1f}s")
print_section("6. 自相关检验 (Durbin-Watson)")
dw = durbin_watson(residuals)
print(f"Durbin-Watson 统计量: {dw:.4f}")
print(f" DW = 2.0 -> 无自相关")

```

```

print(f" DW < 1.5 -> 正自相关")
print(f" DW > 2.5 -> 负自相关")
print(f" 结论: {'可能存在自相关' if abs(dw - 2) > 0.5 else '无严重自相关'}")
t1 = time.time()
print_section("7. 强影响点分析")
n_obs = X_diag_const.shape[0]
n_params = X_diag_const.shape[1]
cooks_threshold = 4 / (n_obs - n_params - 1)
leverage_threshold = 2 * n_params / n_obs
n_influential_cooks = np.sum(cooks > cooks_threshold)
n_high_leverage = np.sum(leverage > leverage_threshold)
print(f"Cook's distance 阈值 (4/(n-p-1)): {cooks_threshold:.6f}")
print(f" 超过阈值的观测: {n_influential_cooks} ({100*n_influential_cooks/n_obs:.1f}%) ")
print(f"杠杆值阈值 (2p/n): {leverage_threshold:.6f}")
print(f" 高杠杆值的观测: {n_high_leverage} ({100*n_high_leverage/n_obs:.1f}%) ")
fig, axes = plt.subplots(1, 3, figsize=(16, 5))
axes[0].scatter(range(len(cooks)), cooks, s=1, alpha=0.4, color=CB_PALETTE[0])
axes[0].axhline(y=cooks_threshold, color="red", linestyle="--",
                label=f"阈值 = {cooks_threshold:.4f}")
axes[0].set_xlabel("观测序号")
axes[0].set_ylabel("Cook's Distance")
axes[0].set_title("Cook's Distance")
axes[0].legend()
axes[1].scatter(leverage, studentized_resid, alpha=0.3, s=2, color=CB_PALETTE[1])
axes[1].axhline(y=0, color="gray", linestyle="--")
axes[1].axvline(x=leverage_threshold, color="red", linestyle="--",
                label=f"杠杆阈值 = {leverage_threshold:.4f}")
axes[1].set_xlabel("杠杆值")
axes[1].set_ylabel("学生化残差")
axes[1].set_title("杠杆值 vs 学生化残差")
axes[1].legend()
top10_idx = np.argsort(cooks)[-10:][::-1]
top10_labels = [f"观测 {i}" for i in top10_idx]
axes[2].barh(range(10), cooks[top10_idx], color=CB_PALETTE[2])
axes[2].set_yticks(range(10))
axes[2].set_yticklabels(top10_labels)
axes[2].set_xlabel("Cook's Distance")
axes[2].set_title("影响最大的前 10 个点")
save_fig("influential_points.png", "04_diagnostics")
influential_df = pd.DataFrame({
    "index": np.arange(len(cooks)),
    "cooks_d": cooks,
    "leverage": leverage,
    "student_resid": studentized_resid,
}).sort_values("cooks_d", ascending=False).head(10)
print("\n影响最大的前 10 个观测: ")
print(influential_df.to_string())
print(f" 强影响点分析用时: {time.time() - t1:.1f}s")
t1 = time.time()

```

```

print_section("8. 稳健标准误 (HC3)")
ols_robust = sm.OLS(y_diag, X_diag_const).fit(cov_type="HC3")
se_comparison = pd.DataFrame({
    "Variable": ols_full.params.index,
    "Coef": ols_full.params.values,
    "SE_Regular": ols_full.bse.values,
    "SE_Robust": ols_robust.bse.values,
    "t_Robust": ols_robust.tvalues.values,
    "p_Robust": ols_robust.pvalues.values,
})
se_comparison["SE_Ratio"] = se_comparison["SE_Robust"] / se_comparison["SE_Regular"]
print("普通 vs 稳健标准误对比 (按比率降序, 前 15): ")
print(se_comparison.sort_values("SE_Ratio", ascending=False).head(15).to_string())
reg_sig = set(ols_full.pvalues[ols_full.pvalues < 0.05].index)
robust_sig = set(ols_robust.pvalues[ols_robust.pvalues < 0.05].index)
changed = reg_sig.symmetric_difference(robust_sig)
print(f"\n使用稳健标准误后显著性发生变化的变量: {len(changed)}")
if len(changed) > 0:
    for var in changed:
        print(f" {var}: p_普通={ols_full.pvalues[var]:.6f}, p_稳健={ols_robust.pvalues[var]:.6f}")
print(f" 稳健标准误用时: {time.time() - t1:.1f}s")
t1 = time.time()
print_section("9. 测试集验证")
X_test_const = sm.add_constant(X_test)
y_pred_test = ols_full.predict(X_test_const)
test_residuals = y_test - y_pred_test
test_r2 = 1 - np.sum(test_residuals**2) / np.sum((y_test - y_test.mean())**2)
test_rmse = np.sqrt(np.mean(test_residuals**2))
test_mae = np.mean(np.abs(test_residuals))
print(f"测试集表现 (对数尺度): ")
print(f" R2:    {test_r2:.4f}")
print(f" RMSE:   {test_rmse:.4f}")
print(f" MAE:    {test_mae:.4f}")
y_test_price = np.exp(y_test)
y_pred_price = np.exp(y_pred_test)
test_rmse_price = np.sqrt(np.mean((y_test_price - y_pred_price)**2))
test_mae_price = np.mean(np.abs(y_test_price - y_pred_price))
print(f"\n反变换回价格尺度: ")
print(f" RMSE:   {test_rmse_price:.1f}")
print(f" MAE:    {test_mae_price:.1f}")
print(f" 测试集验证用时: {time.time() - t1:.1f}s")
print_section("10. 关键数值结果")
norm_stat, norm_pval = stats.normaltest(residuals)
print(f"残差正态性检验 p={norm_pval:.6e}")
VIF > 10: {vif_df['VIF > 10'].sum()} 个变量 | 最大 VIF: {vif_df['VIF'].max():.1f} ({vif_df.iloc
[0]['Variable']})
条件数 (全变量OLS): {np.linalg.cond(X_diag.values):.2e}
条件数 (多项式模型): {cond_poly:.2e}

```

条件数（交互项模型）：{cond_int:.2e}
 异方差：BP p={bp_pval:.6e} | GQ p={gq_pval:.6e}
 自相关：DW={dw:.4f}
 强影响点：{n_influential_cooks}（{100*n_influential_cooks/n_obs:.2f}%）超过 Cook's D 阈值
 使用 HC3 稳健标准误后显著性改变的变量数：{len(changed)}
 测试集：R2={test_r2:.4f}, RMSE={test_rmse_price:.1f}, MAE={test_mae_price:.1f}

05_model_comparison.py ——模型比较

```
import sys
import os
sys.path.insert(0, os.path.dirname(os.path.dirname(os.path.abspath(__file__))))
import warnings
warnings.filterwarnings("ignore")
import time
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import joblib
import statsmodels.api as sm
from sklearn.linear_model import LinearRegression, RidgeCV, LassoCV
from sklearn.preprocessing import PolynomialFeatures, StandardScaler
from sklearn.model_selection import KFold, cross_val_score
from sklearn.pipeline import Pipeline
from sklearn.metrics import r2_score, mean_squared_error, mean_absolute_error
import logging
logging.getLogger("statsmodels").setLevel(logging.WARNING)
from utils import (
    setup_font, print_section, save_fig, save_table, CB_PALETTE
)
setup_font()
SEED = 42
np.random.seed(SEED)
t0 = time.time()
print_section("1. 加载数据")
X_train, X_test, y_train, y_test, feature_cols = joblib.load("output/processed_data.pkl")
X_train_scaled, X_test_scaled, numeric_cols, dummy_cols = joblib.load("output/scaled_data.pkl")
selection_results = joblib.load("output/selection_results.pkl")
consensus_vars = selection_results["consensus_vars"]
print(f"训练集: {X_train.shape[0]:,} 行 x {X_train.shape[1]} 个特征")
print(f"测试集: {X_test.shape[0]:,} 行")
print(f"共识变量: {len(consensus_vars)} 个")
print_section("2. 选择交互/多项式变量")
key_interact_vars = ["dist_center", "Lat", "Lng", "ladderRatio", "DOM"]
key_interact_vars = [v for v in key_interact_vars if v in feature_cols]
print(f"选中的交互变量 ({len(key_interact_vars)} 个): {key_interact_vars}")
remaining_vars = [v for v in feature_cols if v not in key_interact_vars]
print(f"剩余变量: {len(remaining_vars)} 个")
```

```

print_section("3. 拟合模型并评估")
test_metrics = []
cv_metrics = []
models_fitted = {}

def evaluate_model(name, y_true, y_pred, n_params=None):
    r2 = r2_score(y_true, y_pred)
    rmse = np.sqrt(mean_squared_error(y_true, y_pred))
    mae = mean_absolute_error(y_true, y_pred)
    y_true_price = np.exp(y_true)
    y_pred_price = np.exp(y_pred)
    rmse_price = np.sqrt(mean_squared_error(y_true_price, y_pred_price))
    mae_price = mean_absolute_error(y_true_price, y_pred_price)
    result = {
        "Model": name, "R2": round(r2, 4), "RMSE": round(rmse, 4), "MAE": round(mae, 4),
        "RMSE_price": round(rmse_price, 1), "MAE_price": round(mae_price, 1)
    }
    if n_params is not None:
        n = len(y_true)
        rss = np.sum((y_true - y_pred)**2)
        aic = n * np.log(rss / n) + 2 * n_params
        bic = n * np.log(rss / n) + n_params * np.log(n)
        result["AIC"] = round(aic, 1)
        result["BIC"] = round(bic, 1)
    return result

print("拟合 OLS (全变量) ...")
t1 = time.time()
X_train_c = sm.add_constant(X_train)
ols_full = sm.OLS(y_train, X_train_c).fit()
X_test_c = sm.add_constant(X_test)
y_pred_full = ols_full.predict(X_test_c)
res = evaluate_model("OLS (全变量)", y_test, y_pred_full, n_params=X_train_c.shape[1])
test_metrics.append(res)
print(f" R2={res['R2']:.4f}, RMSE={res['RMSE_price']:.1f}, AIC={res['AIC']:.0f} 用时: {time.time() - t1:.1f}s")

print("拟合 Ridge (CV) ...")
t1 = time.time()
ridge_cv = RidgeCV(alphas=np.logspace(-3, 3, 50), cv=5)
ridge_cv.fit(X_train_scaled, y_train)
y_pred_ridge = ridge_cv.predict(X_test_scaled)
res = evaluate_model("Ridge (CV)", y_test, y_pred_ridge)
test_metrics.append(res)
models_fitted["Ridge"] = ridge_cv
print(f" alpha={ridge_cv.alpha_:.4f}, R2={res['R2']:.4f}, RMSE={res['RMSE_price']:.1f} 用时: {time.time() - t1:.1f}s")

print("拟合 LASSO (CV) ...")
t1 = time.time()
lasso_cv = LassoCV(alphas=np.logspace(-4, 0, 100), cv=5, max_iter=10000, random_state=SEED)
lasso_cv.fit(X_train_scaled, y_train)
y_pred_lasso = lasso_cv.predict(X_test_scaled)

```

```

res = evaluate_model("LASSO (CV) ", y_test, y_pred_lasso)
test_metrics.append(res)
models_fitted["LASSO"] = lasso_cv
n_nonzero = np.sum(lasso_cv.coef_ != 0)
print(f" alpha={lasso_cv.alpha_:.6f}, 非零系数={n_nonzero}/{len(feature_cols)}, R2={res['R2']:.4f}
      }, RMSE={res['RMSE_price']:.1f} 用时: {time.time()-t1:.1f}s")
print("拟合 OLS (共识变量) ...")
t1 = time.time()
X_train_sel = sm.add_constant(X_train[consensus_vars])
ols_sel = sm.OLS(y_train, X_train_sel).fit()
X_test_sel = sm.add_constant(X_test[consensus_vars])
y_pred_sel = ols_sel.predict(X_test_sel)
res = evaluate_model("OLS (筛选变量) ", y_test, y_pred_sel, n_params=X_train_sel.shape[1])
test_metrics.append(res)
print(f" R2={res['R2']:.4f}, RMSE={res['RMSE_price']:.1f}, AIC={res['AIC']:.0f} 用时: {time.time()
      (-t1:.1f}s")
print("拟合 OLS + 交互项...")
t1 = time.time()
poly_interact = PolynomialFeatures(degree=2, interaction_only=True, include_bias=False)
X_train_interact = np.hstack([
    X_train[remaining_vars].values,
    poly_interact.fit_transform(X_train[key_interact_vars].values)
])
X_test_interact = np.hstack([
    X_test[remaining_vars].values,
    poly_interact.transform(X_test[key_interact_vars].values)
])
interact_feat_names = remaining_vars + poly_interact.get_feature_names_out(key_interact_vars).
    tolist()
X_train_i_c = sm.add_constant(pd.DataFrame(X_train_interact, columns=interact_feat_names, index=
    X_train.index))
ols_interact = sm.OLS(y_train, X_train_i_c).fit()
X_test_i_c = sm.add_constant(pd.DataFrame(X_test_interact, columns=interact_feat_names, index=
    X_test.index))
y_pred_interact = ols_interact.predict(X_test_i_c)
res = evaluate_model("OLS + 交互项", y_test, y_pred_interact, n_params=X_train_i_c.shape[1])
test_metrics.append(res)
print(f" 特征数={X_train_i_c.shape[1]-1}, R2={res['R2']:.4f}, RMSE={res['RMSE_price']:.1f}, AIC={
    res['AIC']:.0f} 用时: {time.time()-t1:.1f}s")
print("拟合 OLS + 多项式 (degree=2) ...")
t1 = time.time()
poly_full = PolynomialFeatures(degree=2, interaction_only=False, include_bias=False)
X_train_poly = np.hstack([
    X_train[remaining_vars].values,
    poly_full.fit_transform(X_train[key_interact_vars].values)
])
X_test_poly = np.hstack([
    X_test[remaining_vars].values,
    poly_full.transform(X_test[key_interact_vars].values)
])

```

```

])
poly_feat_names = remaining_vars + poly_full.get_feature_names_out(key_interact_vars).tolist()
X_train_p_c = sm.add_constant(pd.DataFrame(X_train_poly, columns=poly_feat_names, index=X_train.
    index))
ols_poly = sm.OLS(y_train, X_train_p_c).fit()
X_test_p_c = sm.add_constant(pd.DataFrame(X_test_poly, columns=poly_feat_names, index=X_test.
    index))
y_pred_poly = ols_poly.predict(X_test_p_c)
res = evaluate_model("OLS + 多项式", y_test, y_pred_poly, n_params=X_train_p_c.shape[1])
test_metrics.append(res)
print(f" 特征数={X_train_p_c.shape[1]-1}, R2={res['R2']:.4f}, RMSE={res['RMSE_price']:.1f}, AIC={
    res['AIC']:.0f} 用时: {time.time()-t1:.1f}s")
print_section("4. 测试集结果汇总")
results_df = pd.DataFrame(test_metrics)
results_df = results_df.sort_values("R2", ascending=False)
save_table(results_df, "model_comparison.csv")
print(results_df.to_string(index=False))
print_section("5. 交叉验证 (5折)")
cv_folds = 5
kf = KFold(n_splits=cv_folds, shuffle=True, random_state=SEED)
cv_results = []
t1 = time.time()
lr = LinearRegression()
cv_r2 = cross_val_score(lr, X_train, y_train, cv=kf, scoring="r2")
cv_rmse = np.sqrt(-cross_val_score(lr, X_train, y_train, cv=kf, scoring="neg_mean_squared_error")
    )
cv_results.append({"Model": "OLS (全变量)", "CV_R2": f"{cv_r2.mean():.4f}±{cv_r2.std():.4f}", "
    CV_RMSE": f"{cv_rmse.mean():.4f}±{cv_rmse.std():.4f}"})
print(f" OLS (全变量): R2={cv_r2.mean():.4f}±{cv_r2.std():.4f}, RMSE={cv_rmse.mean():.4f}±{
    cv_rmse.std():.4f} 用时: {time.time()-t1:.1f}s")
t1 = time.time()
ridge_pipe = Pipeline([("scale", StandardScaler()), ("ridge", RidgeCV(alphas=np.logspace(-3, 3,
    50), cv=3))])
cv_r2 = cross_val_score(ridge_pipe, X_train, y_train, cv=kf, scoring="r2")
cv_rmse = np.sqrt(-cross_val_score(ridge_pipe, X_train, y_train, cv=kf, scoring="
    neg_mean_squared_error"))
cv_results.append({"Model": "Ridge (CV)", "CV_R2": f"{cv_r2.mean():.4f}±{cv_r2.std():.4f}", "
    CV_RMSE": f"{cv_rmse.mean():.4f}±{cv_rmse.std():.4f}"})
print(f" Ridge (CV): R2={cv_r2.mean():.4f}±{cv_r2.std():.4f}, RMSE={cv_rmse.mean():.4f}±{cv_rmse
    .std():.4f} 用时: {time.time()-t1:.1f}s")
t1 = time.time()
lasso_pipe = Pipeline([
    ("scale", StandardScaler()),
    ("lasso", LassoCV(alphas=np.logspace(-4, 0, 100), cv=3, max_iter=10000, random_state=SEED))
])
cv_r2 = cross_val_score(lasso_pipe, X_train, y_train, cv=kf, scoring="r2")
cv_rmse = np.sqrt(-cross_val_score(lasso_pipe, X_train, y_train, cv=kf, scoring="
    neg_mean_squared_error"))

```

```

cv_results.append({"Model": "LASSO (CV) ", "CV_R2": f"{cv_r2.mean():.4f}±{cv_r2.std():.4f}", "
    CV_RMSE": f"{cv_rmse.mean():.4f}±{cv_rmse.std():.4f}"})
print(f" LASSO (CV) : R2={cv_r2.mean():.4f}±{cv_r2.std():.4f}, RMSE={cv_rmse.mean():.4f}±{cv_rmse
    .std():.4f} 用时: {time.time()-t1:.1f}s")
t1 = time.time()
lr_sel = LinearRegression()
cv_r2 = cross_val_score(lr_sel, X_train[consensus_vars], y_train, cv=kf, scoring="r2")
cv_rmse = np.sqrt(-cross_val_score(lr_sel, X_train[consensus_vars], y_train, cv=kf, scoring="
    neg_mean_squared_error"))
cv_results.append({"Model": "OLS (筛选变量) ", "CV_R2": f"{cv_r2.mean():.4f}±{cv_r2.std():.4f}", "
    CV_RMSE": f"{cv_rmse.mean():.4f}±{cv_rmse.std():.4f}"})
print(f" OLS (筛选变量) : R2={cv_r2.mean():.4f}±{cv_r2.std():.4f}, RMSE={cv_rmse.mean():.4f}±{
    cv_rmse.std():.4f} 用时: {time.time()-t1:.1f}s")
t1 = time.time()
cv_r2_list, cv_rmse_list = [], []
for train_idx, val_idx in kf.split(X_train):
    X_tr, X_val = X_train_interact[train_idx], X_train_interact[val_idx]
    y_tr, y_val = y_train.iloc[train_idx], y_train.iloc[val_idx]
    lr_cv = LinearRegression().fit(X_tr, y_tr)
    y_p = lr_cv.predict(X_val)
    cv_r2_list.append(r2_score(y_val, y_p))
    cv_rmse_list.append(np.sqrt(mean_squared_error(y_val, y_p)))
cv_r2 = np.array(cv_r2_list); cv_rmse = np.array(cv_rmse_list)
cv_results.append({"Model": "OLS + 交互项", "CV_R2": f"{cv_r2.mean():.4f}±{cv_r2.std():.4f}", "
    CV_RMSE": f"{cv_rmse.mean():.4f}±{cv_rmse.std():.4f}"})
print(f" OLS + 交互项: R2={cv_r2.mean():.4f}±{cv_r2.std():.4f}, RMSE={cv_rmse.mean():.4f}±{
    cv_rmse.std():.4f} 用时: {time.time()-t1:.1f}s")
t1 = time.time()
cv_r2_list, cv_rmse_list = [], []
for train_idx, val_idx in kf.split(X_train):
    X_tr, X_val = X_train_poly[train_idx], X_train_poly[val_idx]
    y_tr, y_val = y_train.iloc[train_idx], y_train.iloc[val_idx]
    lr_cv = LinearRegression().fit(X_tr, y_tr)
    y_p = lr_cv.predict(X_val)
    cv_r2_list.append(r2_score(y_val, y_p))
    cv_rmse_list.append(np.sqrt(mean_squared_error(y_val, y_p)))
cv_r2 = np.array(cv_r2_list); cv_rmse = np.array(cv_rmse_list)
cv_results.append({"Model": "OLS + 多项式", "CV_R2": f"{cv_r2.mean():.4f}±{cv_r2.std():.4f}", "
    CV_RMSE": f"{cv_rmse.mean():.4f}±{cv_rmse.std():.4f}"})
print(f" OLS + 多项式: R2={cv_r2.mean():.4f}±{cv_r2.std():.4f}, RMSE={cv_rmse.mean():.4f}±{
    cv_rmse.std():.4f} 用时: {time.time()-t1:.1f}s")
cv_df = pd.DataFrame(cv_results)
save_table(cv_df, "cv_comparison.csv")
print("\n交叉验证结果: ")
print(cv_df.to_string(index=False))
print_section("6. 可视化")
fig, axes = plt.subplots(1, 2, figsize=(14, 5))
models_name = results_df["Model"].tolist()
r2_vals = results_df["R2"].tolist()

```

```

rmse_vals = results_df["RMSE_price"].tolist()
colors = [CB_PALETTE[0]] * len(models_name)
best_r2_idx = np.argmax(r2_vals)
colors[best_r2_idx] = CB_PALETTE[1]
axes[0].barh(range(len(models_name)), r2_vals, color=colors[::-1], alpha=0.8)
axes[0].set_yticks(range(len(models_name)))
axes[0].set_yticklabels(models_name[::-1])
axes[0].set_xlabel("R2")
axes[0].set_title("测试集 R2 比较")
for i, v in enumerate(r2_vals):
    axes[0].text(v + 0.002, len(models_name) - 1 - i, f"{v:.4f}", va="center", fontsize=9)
colors_rmse = [CB_PALETTE[0]] * len(models_name)
best_rmse_idx = np.argmin(rmse_vals)
colors_rmse[best_rmse_idx] = CB_PALETTE[1]
axes[1].barh(range(len(models_name)), rmse_vals, color=colors_rmse[::-1], alpha=0.8)
axes[1].set_yticks(range(len(models_name)))
axes[1].set_yticklabels(models_name[::-1])
axes[1].set_xlabel("RMSE (元/m²)")
axes[1].set_title("测试集 RMSE 比较")
for i, v in enumerate(rmse_vals):
    axes[1].text(v + 100, len(models_name) - 1 - i, f"{v:.0f}", va="center", fontsize=9)
save_fig("model_comparison.png", "05_comparison")
print_section("7. 共识变量 OLS 系数图")
coef_series = pd.Series(
    ols_sel.params.drop("const", errors="ignore"),
    index=consensus_vars[:len(ols_sel.params) - 1]
)
top_coefs = coef_series.abs().sort_values(ascending=False).head(15)
top_coef_vals = coef_series[top_coefs.index]
fig, ax = plt.subplots(figsize=(10, 7))
colors_coef = [CB_PALETTE[0] if v > 0 else CB_PALETTE[1] for v in top_coef_vals]
ax.barh(range(len(top_coef_vals)), top_coef_vals.values[::-1],
        color=colors_coef[::-1], alpha=0.8)
ax.set_yticks(range(len(top_coef_vals)))
ax.set_yticklabels(top_coef_vals.index[::-1], fontsize=9)
ax.axvline(x=0, color="gray", linestyle="-", linewidth=0.5)
ax.set_xlabel("系数值")
ax.set_title("共识变量 OLS 模型系数 (前 15 名)")
save_fig("best_model_coefficients.png", "05_comparison")
fig, ax = plt.subplots(figsize=(8, 8))
ax.scatter(y_test, y_pred_sel, alpha=0.1, s=3, color=CB_PALETTE[0])
lims = [min(y_test.min(), y_pred_sel.min()),
        max(y_test.max(), y_pred_sel.max())]
ax.plot(lims, lims, "r--", linewidth=1, label="45° 参考线")
ax.set_xlabel("实际 log(价格)")
ax.set_ylabel("预测 log(价格)")
ax.set_title("预测值 vs 实际值 (共识变量 OLS)")
ax.legend()
r2_sel = r2_score(y_test, y_pred_sel)

```

```

ax.text(0.05, 0.92, f"R2 = {r2_sel:.4f}", transform=ax.transAxes,
        fontsize=12, verticalalignment="top")
save_fig("prediction_vs_actual.png", "05_comparison")
district_coefs = {k: v for k, v in ols_sel.params.items()
                  if k.startswith("district_")}
if district_coefs:
    dist_series = pd.Series(district_coefs).sort_values()
    fig, ax = plt.subplots(figsize=(10, 6))
    colors_dist = [CB_PALETTE[0] if v > 0 else CB_PALETTE[1]
                  for v in dist_series.values]
    ax.barh(range(len(dist_series)), dist_series.values,
            color=colors_dist, alpha=0.8)
    ax.set_yticks(range(len(dist_series)))
    ax.set_yticklabels(dist_series.index, fontsize=9)
    ax.axvline(x=0, color="gray", linestyle="-", linewidth=0.5)
    ax.set_xlabel("系数值（相对基期区域）")
    ax.set_title("区域固定效应系数")
    save_fig("district_coefficients.png", "05_comparison")
print_section("8. 关键数值结果")
print(f"""
最佳模型: {results_df.iloc[0]['Model']}
测试集 R2={results_df.iloc[0]['R2']:.4f}, RMSE={results_df.iloc[0]['RMSE_price']:.1f} 元/m², MAE={
    results_df.iloc[0]['MAE_price']:.1f} 元/m²
各模型表现排名:

```